

Experiment - 1

Data Wrangling I

Perform the following operations using Python on any open source dataset (e.g., data.csv)

1. Import all the required Python Libraries.
2. Locate an open source data from the web (e.g. <https://www.kaggle.com>). Provide a clear description of the data and its source (i.e., URL of the web site).
3. Load the Dataset into pandas data frame.
4. Data Preprocessing: check for missing values in the data using pandas `isnull()`, `describe()` function to get some initial statistics. Provide variable descriptions. Types of variables etc. Check the dimensions of the data frame.
5. Data Formatting and Data Normalization: Summarize the types of variables by checking the data types (i.e., character, numeric, integer, factor, and logical) of the variables in the data set. If variables are not in the correct data type, apply proper type conversions.
6. Turn categorical variables into quantitative variables in Python. In addition to the codes and outputs, explain every operation that you do in the above steps and explain everything that you do to import/read/scrape the data set.

```
In [1]: pip install pandas
```

```
Requirement already satisfied: pandas in c:\users\hp\anaconda3\lib\site-packages (0.25.1)
Requirement already satisfied: pytz>=2017.2 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2019.3)
Requirement already satisfied: python-dateutil>=2.6.1 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2.8.0)
Requirement already satisfied: numpy>=1.13.3 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (1.16.5)
Requirement already satisfied: six>=1.5 in c:\users\hp\anaconda3\lib\site-packages (from python-dateutil>=2.6.1->pandas) (1.12.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import pandas as pd
```

```
In [4]: df = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\1\StudentsPerformance.csv")
```

```
In [5]: print(df)
```

```

      gender race/ethnicity parental level of education      lunch \
0    female      group B      bachelor's degree      standard
1    female      group C      some college      standard
2    female      group B      master's degree      standard
3     male      group A      associate's degree  free/reduced
4     male      group C      some college      standard
..     ...           ...           ...           ...
995  female      group E      master's degree      standard
996   male      group C      high school  free/reduced
997  female      group C      high school  free/reduced
998  female      group D      some college      standard
999  female      group D      some college  free/reduced

      test preparation course  math score  reading score  writing score
0             none           72           72           NaN
1        completed           69           90           88.0
2             none           90           95           93.0
3             none           47           57           44.0
4             none           76           78           75.0
..             ...           ...           ...           ...
995        completed           88           99           95.0
996             none           62           55           55.0
997        completed           59           71           65.0
998        completed           68           78           77.0
999             none           77           86           86.0

```

[1000 rows x 8 columns]

```
In [7]: df.head(15)
```

Out[7]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	NaN
1	female	group C	some college	standard	completed	69	90	88.0
2	female	group B	master's degree	standard	none	90	95	93.0
3	male	group A	associate's degree	free/reduced	none	47	57	44.0
4	male	group C	some college	standard	none	76	78	75.0
5	female	group B	associate's degree	standard	none	71	83	78.0
6	female	group B	some college	standard	completed	88	95	92.0
7	male	group B	some college	free/reduced	none	40	43	39.0
8	male	group D	high school	free/reduced	completed	64	64	67.0
9	female	group B	high school	free/reduced	none	38	60	50.0
10	male	group C	associate's degree	standard	none	58	54	52.0
11	male	group D	associate's degree	standard	none	40	52	43.0
12	female	group B	high school	standard	none	65	81	73.0
13	male	group A	some college	standard	completed	78	72	70.0
14	female	group A	master's degree	standard	none	50	53	58.0

```
In [8]: df.isnull().sum()
```

```
Out[8]: gender                0
race/ethnicity              0
parental level of education  0
lunch                      0
test preparation course     0
math score                  0
reading score                0
writing score                1
dtype: int64
```

```
In [9]: print(df.describe())
```

	math score	reading score	writing score
count	1000.00000	1000.000000	999.000000
mean	66.08900	69.169000	68.048048
std	15.16308	14.600192	15.202102
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.500000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

```
In [11]: df.dtypes
```

```
Out[11]: gender                object
race/ethnicity              object
parental level of education  object
lunch                      object
test preparation course     object
math score                  int64
reading score                int64
writing score                float64
dtype: object
```

```
In [12]: df.dropna(axis=1)
```

```
Out[12]:
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score
0	female	group B	bachelor's degree	standard	none	72	72
1	female	group C	some college	standard	completed	69	90
2	female	group B	master's degree	standard	none	90	95
3	male	group A	associate's degree	free/reduced	none	47	57
4	male	group C	some college	standard	none	76	78
...	...	...	...	...	...	...	...
995	female	group E	master's degree	standard	completed	88	99
996	male	group C	high school	free/reduced	none	62	55
997	female	group C	high school	free/reduced	completed	59	71
998	female	group D	some college	standard	completed	68	78
999	female	group D	some college	free/reduced	none	77	86

1000 rows × 7 columns

```
In [13]: y = df.iloc[:, 0:1]
print(y)
```

```
gender
0    female
1    female
2    female
3     male
4     male
..     ...
995  female
996   male
997  female
998  female
999  female

[1000 rows x 1 columns]
```

```
In [18]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
print(y)
```

```
[0 0 0 1 1 0 0 1 1 0 1 1 0 1 0 0 1 0 1 0 1 0 1 1 1 0 1 0 1 0 0 0 0 1 1 1 0
0 0 1 1 0 0 1 0 1 0 0 0 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 0 1 0 0 1 0 1
1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 0 1 1 1 0 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0
1 1 0 0 1 0 0 0 0 0 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 0 0 0 1 1 0 1 1
0 1 1 1 0 1 1 0 0 1 0 1 1 0 1 1 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 1
1 1 1 1 0 0 1 0 1 0 1 1 1 0 0 0 0 1 0 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1 0 1
0 1 0 0 0 1 1 0 1 1 0 1 1 1 1 0 1 1 1 0 0 1 1 1 1 0 0 1 1 0 0 1 1 0 0 1 0
0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 1 0 0 0 1 1 0 1 1 1 1 1 0 1 1
1 1 1 1 1 1 0 1 0 1 1 1 0 0 0 1 1 0 0 1 0 1 1 0 0 0 0 0 0 1 1 1 0 1 1 1
1 0 0 1 1 0 0 1 0 0 1 1 0 1 0 1 1 0 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 1 1 0 0
1 0 1 0 0 1 0 0 0 1 0 1 1 0 0 0 0 0 0 1 1 0 1 1 0 1 0 0 1 1 0 1 0 0 0 0 1
0 0 1 0 1 1 1 0 1 1 1 1 1 0 0 0 0 1 0 1 1 1 1 1 0 1 0 1 1 1 1 1 0 0 0 0
1 0 1 1 1 1 0 0 0 1 0 1 0 1 0 1 1 1 0 0 1 0 0 1 0 1 0 0 0 0 0 1 1 0 1 1
0 1 1 0 1 1 0 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 1 1 0 1 1 1 0 0 0 0
0 0 1 0 1 1 1 1 1 0 0 0 0 0 1 0 1 0 1 0 1 1 1 1 0 0 0 1 0 1 0 1 1 1 0 1 1
0 0 1 0 1 0 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1
1 0 0 0 1 1 0 0 0 0 0 1 1 1 0 0 0 0 1 0 1 0 0 0 0 1 1 1 0 1 1 1 1 0 1 1 1
0 1 1 0 0 1 1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 1 0 1 1 1 0 0 1 0
0 0 1 1 0 1 0 0 0 0 0 0 1 1 0 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0 1
0 0 1 1 1 1 0 1 0 0 1 0 0 1 0 0 1 0 1 0 1 1 1 0 1 0 1 0 1 0 1 1 0 1 1
1 0 0 0 1 1 1 1 0 1 1 1 1 0 1 0 1 1 0 1 0 0 1 0 1 0 0 1 0 1 1 1 0 0 1 0 0
0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 1 0 1 0 1 1 0 0 0 1 0 0 1 1 1 1 0 1
0 1 0 1 0 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0 1 0 1 0 1 0 0 1 0 1 0 1 1 1 0 1 1
0 0 1 1 0 1 0 1 1 0 0 1 0 1 1 1 1 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 1 0 0 0 1
0 1 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 1 0 0 1 1 0 0 0 0 0 1 0 0 1 1 0 1 0 1
1 1 0 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 0 1 0 1 0 0 1 0 1 1 0 0 1 0 0
0 0 1 0 1 1 0 0 0 1 0 0 0 0 1 1 1 0 0 1 1 0 0 1 0 1 0 0 1 0 0 0 1 0 1 0 0
0]
```

C:\Users\HP\Anaconda3\lib\site-packages\sklearn\preprocessing\label.py:235: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
y = column_or_1d(y, warn=True)
```

```
In [19]: print(df['race/ethnicity'].value_counts())
```

```
group C    319
group D    262
group B    190
group E    140
group A     89
Name: race/ethnicity, dtype: int64
```

```
In [20]: df_Lunch = pd.get_dummies(df['lunch'])
df_new = pd.concat([df, df_Lunch], axis=1)
print(df_new)
```

	gender	race/ethnicity	parental level of education	lunch	\
0	female	group B	bachelor's degree	standard	
1	female	group C	some college	standard	
2	female	group B	master's degree	standard	
3	male	group A	associate's degree	free/reduced	
4	male	group C	some college	standard	
..	...	...	...	...	
995	female	group E	master's degree	standard	
996	male	group C	high school	free/reduced	
997	female	group C	high school	free/reduced	
998	female	group D	some college	standard	
999	female	group D	some college	free/reduced	
	test preparation course	math score	reading score	writing score	\
0	none	72	72	NaN	
1	completed	69	90	88.0	
2	none	90	95	93.0	
3	none	47	57	44.0	
4	none	76	78	75.0	
..	...	...	...	...	
995	completed	88	99	95.0	
996	none	62	55	55.0	
997	completed	59	71	65.0	
998	completed	68	78	77.0	
999	none	77	86	86.0	
	free/reduced	standard			
0	0	1			
1	0	1			
2	0	1			
3	1	0			
4	0	1			
..	...	...			
995	0	1			
996	1	0			
997	1	0			
998	0	1			
999	1	0			

[1000 rows x 10 columns]

```
In [ ]:
```


Experiment - 2

Data Wrangling II

Create an “Academic performance” dataset of students and perform the following operations using Python.

1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them.
2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them.
3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution. Reason and document your approach properly.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [2]: import pandas as pd
```

```
In [3]: df = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\2\AcademicPerformance.csv")
```

```
In [4]: print(df)
```

	gender	race/ethnicity	parental level of education	lunch	\
0	female	group B	bachelor's degree	standard	
1	female	group C	some college	standard	
2	female	group B	master's degree	standard	
3	male	group A	associate's degree	free/reduced	
4	male	group C	some college	standard	
...	...	...	...	...	
2235	NaN	NaN	NaN	NaN	
2236	NaN	NaN	NaN	NaN	
2237	NaN	NaN	NaN	NaN	
2238	NaN	NaN	NaN	NaN	
2239	NaN	NaN	NaN	NaN	

	test preparation course	Year_Birth	math score	reading score	\
0	none	1970.0	72.0	72	
1	completed	1961.0	NaN	na	
2	none	1958.0	90.0	95	
3	none	1967.0	NaN	NaN	
4	none	1989.0	76.0	78	
...	...	...	...	...	
2235	NaN	NaN	NaN	NaN	
2236	NaN	NaN	NaN	NaN	
2237	NaN	NaN	NaN	NaN	
2238	NaN	NaN	NaN	NaN	
2239	NaN	NaN	NaN	NaN	

	writing score	Dt_Admission	College_Fees
0	74	6/16/14	\$84,835.00
1	A	6/15/14	\$57,091.00
2	93	5/13/14	\$67,267.00
3	44	05-11-2014	\$32,474.00
4	75	04-08-2014	\$21,474.00
...	...	...	...
2235	NaN	NaN	NaN
2236	NaN	NaN	NaN
2237	NaN	NaN	NaN
2238	NaN	NaN	NaN
2239	NaN	NaN	NaN

[2240 rows x 11 columns]

```
In [5]: print(df['math score'])
```

0	72.0
1	NaN
2	90.0
3	NaN
4	76.0
...	
2235	NaN
2236	NaN
2237	NaN
2238	NaN
2239	NaN

Name: math score, Length: 2240, dtype: float64

```
In [6]: print(df['math score'].isnull())
```

```
0      False
1       True
2      False
3       True
4      False
...
2235    True
2236    True
2237    True
2238    True
2239    True
Name: math score, Length: 2240, dtype: bool
```

```
In [7]: print(df['reading score'])
```

```
0      72
1      na
2      95
3     NaN
4      78
...
2235   NaN
2236   NaN
2237   NaN
2238   NaN
2239   NaN
Name: reading score, Length: 2240, dtype: object
```

```
In [8]: print(df['reading score'].isnull())
```

```
0      False
1      False
2      False
3       True
4      False
...
2235    True
2236    True
2237    True
2238    True
2239    True
Name: reading score, Length: 2240, dtype: bool
```

```
In [9]: missing_values = ["n/a", "na", "--"]
df = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\2\AcademicPerformance.csv", na_v
```

```
In [10]: print(df['reading score'])
```

```
0      72.0
1      NaN
2      95.0
3      NaN
4      78.0
...
2235    NaN
2236    NaN
2237    NaN
2238    NaN
2239    NaN
Name: reading score, Length: 2240, dtype: float64
```

```
In [11]: print(df['reading score'].isnull())
```

```
0      False
1       True
2      False
3       True
4      False
...
2235     True
2236     True
2237     True
2238     True
2239     True
Name: reading score, Length: 2240, dtype: bool
```

```
In [12]: dataset = [11,41,20,3,101,55,68,97,99,6]
```

```
In [13]: sorted(dataset)
```

```
Out[13]: [3, 6, 11, 20, 41, 55, 68, 97, 99, 101]
```

```
In [14]: quantile1, quantile3 = np.percentile(dataset, [25,75])
```

```
In [15]: print(quantile1, quantile3)
```

```
13.25 89.75
```

```
In [16]: iqr_value = (quantile3 - quantile1)
```

```
In [17]: print(iqr_value)
```

```
76.5
```

```
In [18]: lower_bound_value = quantile1 - (1.5*iqr_value)
```

```
In [19]: upper_bound_value = quantile3 + (1.5*iqr_value)
```

```
In [20]: print(lower_bound_value, upper_bound_value)
```

```
-101.5 204.5
```

```
In [21]: from datetime import date
df['age'] = date.today().year - df['Year_Birth']
```

```
In [22]: df['Year'] = pd.DatetimeIndex(df['Dt_Admission']).year
df['E_L'] = date.today().year - df['Year']
```

```
In [23]: df.head(5)
```

Out[23]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	Year_Birth	math score	reading score	writing score	Dt_Admission
0	female	group B	bachelor's degree	standard	none	1970.0	72.0	72.0	74	6/11/2010
1	female	group C	some college	standard	completed	1961.0	NaN	NaN	A	6/11/2010
2	female	group B	master's degree	standard	none	1958.0	90.0	95.0	93	5/11/2010
3	male	group A	associate's degree	free/reduced	none	1967.0	NaN	NaN	44	05-11-2010
4	male	group C	some college	standard	none	1989.0	76.0	78.0	75	04-08-2010

```
In [24]: df['Fees$'] = df['College_Fees'].str.replace(',', '').str.replace('$', '').str.replace(' ', '')
df['Fees_M$'] = df['Fees$'].apply(lambda X:round(X/1000000))
```

```
In [25]: df.head(5)
```

Out[25]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	Year_Birth	math score	reading score	writing score	Dt_Admission
0	female	group B	bachelor's degree	standard	none	1970.0	72.0	72.0	74	6/11/2010
1	female	group C	some college	standard	completed	1961.0	NaN	NaN	A	6/11/2010
2	female	group B	master's degree	standard	none	1958.0	90.0	95.0	93	5/11/2010
3	male	group A	associate's degree	free/reduced	none	1967.0	NaN	NaN	44	05-11-2010
4	male	group C	some college	standard	none	1989.0	76.0	78.0	75	04-08-2010

```
In [ ]:
```

Experiment - 3

Descriptive Statistics - Measures of Central Tendency and variability

Perform the following operations on any open source dataset (e.g., data.csv)

1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.) with numeric variables grouped by one of the qualitative (categorical) variable. For example, if your categorical variable is age groups and quantitative variable is income, then provide summary statistics of income grouped by the age groups. Create a list that contains a numeric value for each response to the categorical variable.

2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris-versicolor' of iris.csv dataset.

Provide the codes with outputs and explain everything that you do in this step.

```
In [1]: import numpy

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.mean(speed)

print(x)
```

89.76923076923077

```
In [2]: import numpy

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = numpy.median(speed)

print(x)
```

87.0

```
In [7]: import numpy

speed = [99,86,87,88,86,103,87,94,78,77,85,86]

x = numpy.median(speed)

print(x)
```

86.5

```
In [8]: from scipy import stats

speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]

x = stats.mode(speed)

print(x)
```

ModeResult(mode=array([86]), count=array([3]))

```
In [9]: n_num = [1, 2, 3, 4, 5]
n = len(n_num)

get_sum = sum(n_num)
mean = get_sum/n

print("Mean / Average is : " + str(mean))
```

Mean / Average is : 3.0

```
In [10]: n_num = [1, 2, 3, 4, 5]
n = len(n_num)
n_num.sort()

if n % 2 == 0:
    median1 = n_num[n//2]
    median2 = n_num[n//2-1]
    median = (median1 + median2)/2
else:
    median = n_num[n//2]
print("Median is : " + str(median))
```

Median is : 3

```
In [16]: from collections import Counter

n_num = [1, 2, 3, 4, 5, 5]
n = len(n_num)

data = Counter(n_num)
get_mode = dict(data)
mode = [k for k, v in get_mode.items() if v == max(list(data.values()))]

if len(mode) == n:
    get_mode = "No mode found"
else:
    get_mode = "Mode is/are : " + ', '.join(map(str, mode))
print(get_mode)
```

Mode is/are : 5

```
In [17]: import pandas as pd

df = pd.DataFrame({'A' : ['a', 'b', 'c', 'c', 'a', 'b'],
                  'B' : [0, 1, 1, 0, 1, 0]}, dtype = "category")
df.dtypes
```

```
Out[17]: A    category
B    category
dtype: object
```

```
In [18]: print(df)

print(df.groupby(['A']). count().reset_index())
```

```
   A  B
0  a  0
1  b  1
2  c  1
3  c  0
4  a  1
5  b  0
   A  B
0  a  2
1  b  2
2  c  2
```

```
In [19]: import pandas as pd

df = pd.DataFrame({'A' : ['a', 'b', 'c', 'c', 'a', 'b'],
                   'B' : [0, 1, 1, 0, 1, 0],
                   'C' : [7, 8, 9, 5, 3, 6]})

df['A'] = df['A'].astype('category')

print(df)

print(df.groupby(['A', 'B']).mean().reset_index())
```

```
   A  B  C
0  a  0  7
1  b  1  8
2  c  1  9
3  c  0  5
4  a  1  3
5  b  0  6
   A  B  C
0  a  0  7
1  a  1  3
2  b  0  6
3  b  1  8
4  c  0  5
5  c  1  9
```

```
In [22]: import pandas as pd

data = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\3\Iris.csv")

print('Iris-setosa')
setosa= data['Species'] == 'Iris-setosa'
print(data[setosa].describe())

print('\nIris-versicolor')
versicolor= data['Species'] == 'Iris-versicolor'
print(data[versicolor].describe())

print('\nIris-virginica')
virginica = data['Species'] == 'Iris-virginica'
print (data[virginica].describe())
```

Iris-setosa

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	25.50000	5.00600	3.41800	1.46400	0.24400
std	14.57738	0.35249	0.38102	0.17351	0.10721
min	1.00000	4.30000	2.30000	1.00000	0.10000
25%	13.25000	4.80000	3.12500	1.40000	0.20000
50%	25.50000	5.00000	3.40000	1.50000	0.20000
75%	37.75000	5.20000	3.67500	1.57500	0.30000
max	50.00000	5.80000	4.40000	1.90000	0.60000

Iris-versicolor

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	75.50000	5.93600	2.77000	4.26000	1.32600
std	14.57738	0.51617	0.31379	0.46991	0.19775
min	51.00000	4.90000	2.00000	3.00000	1.00000
25%	63.25000	5.60000	2.52500	4.00000	1.20000
50%	75.50000	5.90000	2.80000	4.35000	1.30000
75%	87.75000	6.30000	3.00000	4.60000	1.50000
max	100.00000	7.00000	3.40000	5.10000	1.80000

Iris-virginica

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	125.50000	6.58800	2.97400	5.55200	2.02600
std	14.57738	0.63588	0.32249	0.55189	0.27465
min	101.00000	4.90000	2.20000	4.50000	1.40000
25%	113.25000	6.22500	2.80000	5.10000	1.80000
50%	125.50000	6.50000	3.00000	5.55000	2.00000
75%	137.75000	6.90000	3.17500	5.87500	2.30000
max	150.00000	7.90000	3.80000	6.90000	2.50000

```
In [ ]:
```

Experiment - 4

Data Analytics I

Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features

```
In [2]: import pandas as pd
import numpy as np
from sklearn import linear_model
from sklearn.model_selection import train_test_split
```

```
In [3]: from sklearn.datasets import load_boston
boston = load_boston()
print(boston)
```

```
{'data': array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
 4.9800e+00],
 [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
 9.1400e+00],
 [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
 4.0300e+00],
 ...,
 [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
 5.6400e+00],
 [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
 6.4800e+00],
 [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
 7.8800e+00]]), 'target': array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 2
7.1, 16.5, 18.9, 15. ,
 18.9, 21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 18.2, 13.6, 19.6,
 15.2, 14.5, 15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 14.5, 13.2,
 13.1, 13.5, 18.9, 20. , 21. , 24.7, 30.8, 34.9, 26.6, 25.3, 24.7,
 21.2, 19.3, 20. , 16.6, 14.4, 19.4, 19.7, 20.5, 25. , 23.4, 18.9,
 35.4, 24.7, 31.6, 23.3, 19.6, 18.7, 16. , 22.2, 25. , 33. , 23.5,
 19.4, 22. , 17.4, 20.9, 24.2, 21.7, 22.8, 23.4, 24.1, 21.4, 20. ,
 20.8, 21.2, 20.3, 28. , 23.9, 24.8, 22.9, 23.9, 26.6, 22.5, 22.2,
 23.6, 28.7, 22.6, 22. , 22.9, 25. , 20.6, 28.4, 21.4, 38.7, 43.8,
 33.2, 27.5, 26.5, 18.6, 19.3, 20.1, 19.5, 19.5, 20.4, 19.8, 19.4,
 21.7, 22.8, 18.8, 18.7, 18.5, 18.3, 21.2, 19.2, 20.4, 19.3, 22. ,
 20.3, 20.5, 17.3, 18.8, 21.4, 15.7, 16.2, 18. , 14.3, 19.2, 19.6,
 23. , 18.4, 15.6, 18.1, 17.4, 17.1, 13.3, 17.8, 14. , 14.4, 13.4,
 15.6, 11.8, 13.8, 15.6, 14.6, 17.8, 15.4, 21.5, 19.6, 15.3, 19.4,
 17. , 15.6, 13.1, 41.3, 24.3, 23.3, 27. , 50. , 50. , 50. , 22.7,
 25. , 50. , 23.8, 23.8, 22.3, 17.4, 19.1, 23.1, 23.6, 22.6, 29.4,
 23.2, 24.6, 29.9, 37.2, 39.8, 36.2, 37.9, 32.5, 26.4, 29.6, 50. ,
 32. , 29.8, 34.9, 37. , 30.5, 36.4, 31.1, 29.1, 50. , 33.3, 30.3,
 34.6, 34.9, 32.9, 24.1, 42.3, 48.5, 50. , 22.6, 24.4, 22.5, 24.4,
 20. , 21.7, 19.3, 22.4, 28.1, 23.7, 25. , 23.3, 28.7, 21.5, 23. ,
 26.7, 21.7, 27.5, 30.1, 44.8, 50. , 37.6, 31.6, 46.7, 31.5, 24.3,
 31.7, 41.7, 48.3, 29. , 24. , 25.1, 31.5, 23.7, 23.3, 22. , 20.1,
 22.2, 23.7, 17.6, 18.5, 24.3, 20.5, 24.5, 26.2, 24.4, 24.8, 29.6,
 42.8, 21.9, 20.9, 44. , 50. , 36. , 30.1, 33.8, 43.1, 48.8, 31. ,
 36.5, 22.8, 30.7, 50. , 43.5, 20.7, 21.1, 25.2, 24.4, 35.2, 32.4,
 32. , 33.2, 33.1, 29.1, 35.1, 45.4, 35.4, 46. , 50. , 32.2, 22. ,
 20.1, 23.2, 22.3, 24.8, 28.5, 37.3, 27.9, 23.9, 21.7, 28.6, 27.1,
 20.3, 22.5, 29. , 24.8, 22. , 26.4, 33.1, 36.1, 28.4, 33.4, 28.2,
 22.8, 20.3, 16.1, 22.1, 19.4, 21.6, 23.8, 16.2, 17.8, 19.8, 23.1,
 21. , 23.8, 23.1, 20.4, 18.5, 25. , 24.6, 23. , 22.2, 19.3, 22.6,
 19.8, 17.1, 19.4, 22.2, 20.7, 21.1, 19.5, 18.5, 20.6, 19. , 18.7,
 32.7, 16.5, 23.9, 31.2, 17.5, 17.2, 23.1, 24.5, 26.6, 22.9, 24.1,
 18.6, 30.1, 18.2, 20.6, 17.8, 21.7, 22.7, 22.6, 25. , 19.9, 20.8,
 16.8, 21.9, 27.5, 21.9, 23.1, 50. , 50. , 50. , 50. , 50. , 13.8,
 13.8, 15. , 13.9, 13.3, 13.1, 10.2, 10.4, 10.9, 11.3, 12.3, 8.8,
 7.2, 10.5, 7.4, 10.2, 11.5, 15.1, 23.2, 9.7, 13.8, 12.7, 13.1,
 12.5, 8.5, 5. , 6.3, 5.6, 7.2, 12.1, 8.3, 8.5, 5. , 11.9,
 27.9, 17.2, 27.5, 15. , 17.2, 17.9, 16.3, 7. , 7.2, 7.5, 10.4,
 8.8, 8.4, 16.7, 14.2, 20.8, 13.4, 11.7, 8.3, 10.2, 10.9, 11. ,
 9.5, 14.5, 14.1, 16.1, 14.3, 11.7, 13.4, 9.6, 8.7, 8.4, 12.8,
 10.5, 17.1, 18.4, 15.4, 10.8, 11.8, 14.9, 12.6, 14.1, 13. , 13.4,
 15.2, 16.1, 17.8, 14.9, 14.1, 12.7, 13.5, 14.9, 20. , 16.4, 17.7,
 19.5, 20.2, 21.4, 19.9, 19. , 19.1, 19.1, 20.1, 19.9, 19.6, 23.2,
 29.8, 13.8, 13.3, 16.7, 12. , 14.6, 21.4, 23. , 23.7, 25. , 21.8,
 20.6, 21.2, 19.1, 20.6, 15.2, 7. , 8.1, 13.6, 20.1, 21.8, 24.5,
```

```

23.1, 19.7, 18.3, 21.2, 17.5, 16.8, 22.4, 20.6, 23.9, 22. , 11.9]], 'feature
_names': array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',
'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='<U7'), 'DESCR': ".. _boston_datase
t:\n\nBoston house prices dataset\n-----\n\n**Data Set Charac
teristics:** \n\n      :Number of Instances: 506 \n\n      :Number of Attributes: 13 n
umeric/categorical predictive. Median Value (attribute 14) is usually the target.\n
\n      :Attribute Information (in order):\n          - CRIM      per capita crime rate
by town\n          - ZN      proportion of residential land zoned for lots over 25,0
00 sq.ft.\n          - INDUS  proportion of non-retail business acres per town\n
- CHAS  Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)\n
- NOX    nitric oxides concentration (parts per 10 million)\n          - RM      a
verage number of rooms per dwelling\n          - AGE      proportion of owner-occupe
d units built prior to 1940\n          - DIS      weighted distances to five Boston e
mployment centres\n          - RAD      index of accessibility to radial highways\n
- TAX     full-value property-tax rate per $10,000\n          - PTRATIO  pupil-teach
er ratio by town\n          - B      1000(Bk - 0.63)^2 where Bk is the proportion o
f blacks by town\n          - LSTAT   % lower status of the population\n          - ME
DV      Median value of owner-occupied homes in $1000's\n\n      :Missing Attribute Va
lues: None\n\n      :Creator: Harrison, D. and Rubinfeld, D.L.\n\nThis is a copy of U
CI ML housing dataset.\nhttps://archive.ics.uci.edu/ml/machine-learning-databases/h
ousing/\n\n\nThis dataset was taken from the StatLib library which is maintained at
Carnegie Mellon University.\n\nThe Boston house-price data of Harrison, D. and Rubi
nfeld, D.L. 'Hedonic\nprices and the demand for clean air', J. Environ. Economics &
Management,\nvol.5, 81-102, 1978.  Used in Belsley, Kuh & Welsch, 'Regression diag
nostics\n...', Wiley, 1980.  N.B. Various transformations are used in the table on
\npages 244-261 of the latter.\n\nThe Boston house-price data has been used in many
machine learning papers that address regression\nproblems.  \n      \n.. topic:: Re
ferences\n\n      - Belsley, Kuh & Welsch, 'Regression diagnostics: Identifying Influe
ntial Data and Sources of Collinearity', Wiley, 1980. 244-261.\n      - Quinlan,R. (19
93). Combining Instance-Based and Model-Based Learning. In Proceedings on the Tenth
International Conference of Machine Learning, 236-243, University of Massachusetts,
Amherst. Morgan Kaufmann.\n", 'filename': 'C:\\Users\\HP\\Anaconda3\\lib\\site-pack
ages\\sklearn\\datasets\\data\\boston_house_prices.csv'}

```

In [4]: boston.feature_names

Out[4]: array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD',
'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='<U7')

```
In [5]: boston.target
```

```
Out[5]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
18.9, 21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 18.2, 13.6, 19.6,
15.2, 14.5, 15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 14.5, 13.2,
13.1, 13.5, 18.9, 20. , 21. , 24.7, 30.8, 34.9, 26.6, 25.3, 24.7,
21.2, 19.3, 20. , 16.6, 14.4, 19.4, 19.7, 20.5, 25. , 23.4, 18.9,
35.4, 24.7, 31.6, 23.3, 19.6, 18.7, 16. , 22.2, 25. , 33. , 23.5,
19.4, 22. , 17.4, 20.9, 24.2, 21.7, 22.8, 23.4, 24.1, 21.4, 20. ,
20.8, 21.2, 20.3, 28. , 23.9, 24.8, 22.9, 23.9, 26.6, 22.5, 22.2,
23.6, 28.7, 22.6, 22. , 22.9, 25. , 20.6, 28.4, 21.4, 38.7, 43.8,
33.2, 27.5, 26.5, 18.6, 19.3, 20.1, 19.5, 19.5, 20.4, 19.8, 19.4,
21.7, 22.8, 18.8, 18.7, 18.5, 18.3, 21.2, 19.2, 20.4, 19.3, 22. ,
20.3, 20.5, 17.3, 18.8, 21.4, 15.7, 16.2, 18. , 14.3, 19.2, 19.6,
23. , 18.4, 15.6, 18.1, 17.4, 17.1, 13.3, 17.8, 14. , 14.4, 13.4,
15.6, 11.8, 13.8, 15.6, 14.6, 17.8, 15.4, 21.5, 19.6, 15.3, 19.4,
17. , 15.6, 13.1, 41.3, 24.3, 23.3, 27. , 50. , 50. , 50. , 22.7,
25. , 50. , 23.8, 23.8, 22.3, 17.4, 19.1, 23.1, 23.6, 22.6, 29.4,
23.2, 24.6, 29.9, 37.2, 39.8, 36.2, 37.9, 32.5, 26.4, 29.6, 50. ,
32. , 29.8, 34.9, 37. , 30.5, 36.4, 31.1, 29.1, 50. , 33.3, 30.3,
34.6, 34.9, 32.9, 24.1, 42.3, 48.5, 50. , 22.6, 24.4, 22.5, 24.4,
20. , 21.7, 19.3, 22.4, 28.1, 23.7, 25. , 23.3, 28.7, 21.5, 23. ,
26.7, 21.7, 27.5, 30.1, 44.8, 50. , 37.6, 31.6, 46.7, 31.5, 24.3,
31.7, 41.7, 48.3, 29. , 24. , 25.1, 31.5, 23.7, 23.3, 22. , 20.1,
22.2, 23.7, 17.6, 18.5, 24.3, 20.5, 24.5, 26.2, 24.4, 24.8, 29.6,
42.8, 21.9, 20.9, 44. , 50. , 36. , 30.1, 33.8, 43.1, 48.8, 31. ,
36.5, 22.8, 30.7, 50. , 43.5, 20.7, 21.1, 25.2, 24.4, 35.2, 32.4,
32. , 33.2, 33.1, 29.1, 35.1, 45.4, 35.4, 46. , 50. , 32.2, 22. ,
20.1, 23.2, 22.3, 24.8, 28.5, 37.3, 27.9, 23.9, 21.7, 28.6, 27.1,
20.3, 22.5, 29. , 24.8, 22. , 26.4, 33.1, 36.1, 28.4, 33.4, 28.2,
22.8, 20.3, 16.1, 22.1, 19.4, 21.6, 23.8, 16.2, 17.8, 19.8, 23.1,
21. , 23.8, 23.1, 20.4, 18.5, 25. , 24.6, 23. , 22.2, 19.3, 22.6,
19.8, 17.1, 19.4, 22.2, 20.7, 21.1, 19.5, 18.5, 20.6, 19. , 18.7,
32.7, 16.5, 23.9, 31.2, 17.5, 17.2, 23.1, 24.5, 26.6, 22.9, 24.1,
18.6, 30.1, 18.2, 20.6, 17.8, 21.7, 22.7, 22.6, 25. , 19.9, 20.8,
16.8, 21.9, 27.5, 21.9, 23.1, 50. , 50. , 50. , 50. , 50. , 13.8,
13.8, 15. , 13.9, 13.3, 13.1, 10.2, 10.4, 10.9, 11.3, 12.3, 8.8,
7.2, 10.5, 7.4, 10.2, 11.5, 15.1, 23.2, 9.7, 13.8, 12.7, 13.1,
12.5, 8.5, 5. , 6.3, 5.6, 7.2, 12.1, 8.3, 8.5, 5. , 11.9,
27.9, 17.2, 27.5, 15. , 17.2, 17.9, 16.3, 7. , 7.2, 7.5, 10.4,
8.8, 8.4, 16.7, 14.2, 20.8, 13.4, 11.7, 8.3, 10.2, 10.9, 11. ,
9.5, 14.5, 14.1, 16.1, 14.3, 11.7, 13.4, 9.6, 8.7, 8.4, 12.8,
10.5, 17.1, 18.4, 15.4, 10.8, 11.8, 14.9, 12.6, 14.1, 13. , 13.4,
15.2, 16.1, 17.8, 14.9, 14.1, 12.7, 13.5, 14.9, 20. , 16.4, 17.7,
19.5, 20.2, 21.4, 19.9, 19. , 19.1, 19.1, 20.1, 19.9, 19.6, 23.2,
29.8, 13.8, 13.3, 16.7, 12. , 14.6, 21.4, 23. , 23.7, 25. , 21.8,
20.6, 21.2, 19.1, 20.6, 15.2, 7. , 8.1, 13.6, 20.1, 21.8, 24.5,
23.1, 19.7, 18.3, 21.2, 17.5, 16.8, 22.4, 20.6, 23.9, 22. , 11.9])
```

```
In [6]: x = pd.DataFrame(boston.data, columns = boston.feature_names)
y = pd.DataFrame(boston.target)
```

```
In [7]: x.head(10)
```

Out[7]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33
5	0.02985	0.0	2.18	0.0	0.458	6.430	58.7	6.0622	3.0	222.0	18.7	394.12	5.21
6	0.08829	12.5	7.87	0.0	0.524	6.012	66.6	5.5605	5.0	311.0	15.2	395.60	12.43
7	0.14455	12.5	7.87	0.0	0.524	6.172	96.1	5.9505	5.0	311.0	15.2	396.90	19.15
8	0.21124	12.5	7.87	0.0	0.524	5.631	100.0	6.0821	5.0	311.0	15.2	386.63	29.93
9	0.17004	12.5	7.87	0.0	0.524	6.004	85.9	6.5921	5.0	311.0	15.2	386.71	17.10

```
In [8]: y.head(10)
```

Out[8]:

	0
0	24.0
1	21.6
2	34.7
3	33.4
4	36.2
5	28.7
6	22.9
7	27.1
8	16.5
9	18.9

```
In [9]: reg = linear_model.LinearRegression()
```

```
In [10]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=
```

```
In [11]: reg.fit(x_train, y_train)
```

Out[11]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

```
In [12]: print(reg.coef_)
```

```
[[-1.13055924e-01  3.01104641e-02  4.03807204e-02  2.78443820e+00
 -1.72026334e+01  4.43883520e+00 -6.29636221e-03 -1.44786537e+00
  2.62429736e-01 -1.06467863e-02 -9.15456240e-01  1.23513347e-02
 -5.08571424e-01]]
```

```
In [13]: y_pred = reg.predict(x_test)
print(y_pred)
```

```
[[28.99672362]
 [36.02556534]
 [14.81694405]
 [25.03197915]
 [18.76987992]
 [23.25442929]
 [17.66253818]
 [14.34119   ]
 [23.01320703]
 [20.63245597]
 [24.90850512]
 [18.63883645]
 [-6.08842184]
 [21.75834668]
 [19.23922576]
 [26.19319733]
 [20.64773313]
 [ 5.79472718]
 [40.50033966]
 [17.61289074]
 [27.24909479]
 [30.06625441]
 [11.34179277]
 [24.16077616]
 [17.86058499]
 [15.83609765]
 [22.78148106]
 [14.57704449]
 [22.43626052]
 [19.19631835]
 [22.43383455]
 [25.21979081]
 [25.93909562]
 [17.70162434]
 [16.76911711]
 [16.95125411]
 [31.23340153]
 [20.13246729]
 [23.76579011]
 [24.6322925 ]
 [13.94204955]
 [32.25576301]
 [42.67251161]
 [17.32745046]
 [27.27618614]
 [16.99310991]
 [14.07009109]
 [25.90341861]
 [20.29485982]
 [29.95339638]
 [21.28860173]
 [34.34451856]
 [16.04739105]
 [26.22562412]
 [39.53939798]
 [22.57950697]
 [18.84531367]
 [32.72531661]
 [25.0673037 ]]
```

```
[12.88628956]
[22.68221908]
[30.48287757]
[31.52626806]
[15.90148607]
[20.22094826]
[16.71089812]
[20.52384893]
[25.96356264]
[30.61607978]
[11.59783023]
[20.51232627]
[27.48111878]
[11.01962332]
[15.68096344]
[23.79316251]
[ 6.19929359]
[21.6039073 ]
[41.41377225]
[18.76548695]
[ 8.87931901]
[20.83076916]
[13.25620627]
[20.73963699]
[ 9.36482222]
[23.22444271]
[31.9155003 ]
[19.10228271]
[25.51579303]
[29.04256769]
[20.14358566]
[25.5859787 ]
[ 5.70159447]
[20.09474756]
[14.95069156]
[12.50395648]
[20.72635294]
[24.73957161]
[-0.164237 ]
[13.68486682]
[16.18359697]
[22.27621999]
[24.47902364]]
```

In []:

Experiment - 5

Data Analytics II

1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [4]: dataset = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\5\Social_Network_Ads.csv")
```

```
In [5]: dataset.head()
```

Out[5]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
In [6]: dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
User ID      400 non-null int64
Gender       400 non-null object
Age          400 non-null int64
EstimatedSalary  400 non-null int64
Purchased    400 non-null int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
In [7]: dataset.isnull().sum()
```

```
Out[7]: User ID      0
Gender      0
Age         0
EstimatedSalary  0
Purchased   0
dtype: int64
```

```
In [8]: dataset.shape
```

```
Out[8]: (400, 5)
```

```
In [10]: x = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
```

In [11]: `print(x)`

```
[[ 19 19000]
 [ 35 20000]
 [ 26 43000]
 [ 27 57000]
 [ 19 76000]
 [ 27 58000]
 [ 27 84000]
 [ 32 150000]
 [ 25 33000]
 [ 35 65000]
 [ 26 80000]
 [ 26 52000]
 [ 20 86000]
 [ 32 18000]
 [ 18 82000]
 [ 29 80000]
 [ 47 25000]
 [ 45 26000]
 [ 46 28000]
 [ 46 28000]]
```

In [12]: `from sklearn.model_selection import train_test_split`
`x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.30, random_state=0)`

In [13]: `from sklearn.preprocessing import StandardScaler`
`sc = StandardScaler()`
`x_train = sc.fit_transform(x_train)`
`x_test = sc.transform(x_test)`

In [14]: `from sklearn.linear_model import LogisticRegression`
`classifier = LogisticRegression(random_state = 0)`
`classifier.fit(x_train, y_train)`

C:\Users\HP\Anaconda3\lib\site-packages\sklearn\linear_model\logistic.py:432: FutureWarning: Default solver will be changed to 'lbfgs' in 0.22. Specify a solver to silence this warning.
FutureWarning)

Out[14]: `LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True, intercept_scaling=1, l1_ratio=None, max_iter=100, multi_class='warn', n_jobs=None, penalty='l2', random_state=0, solver='warn', tol=0.0001, verbose=0, warm_start=False)`

In [15]: `y_pred = classifier.predict(x_test)`

In [16]: `print(y_pred)`

```
[0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0
 0 0 1 0 1 1 1 1 0 0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0
 0 0 1 1 1 1 0 1 1]
```

In [17]: `print(y_test)`

```
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1
 0 0 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 0 0 0
 0 1 1 1 0 1 0 0 1]
```

In [18]: `from sklearn.metrics import confusion_matrix`
`cm = confusion_matrix(y_test, y_pred)`
`print(cm)`

```
[[74  5]
 [10 31]]
```

In [19]: `Accuracy=(74+31)/120`
`Accuracy`

Out[19]: 0.875

In [20]: `Error_rate=(5+10)/120`
`Error_rate`

Out[20]: 0.125

In [21]: `from sklearn.metrics import precision_score, recall_score`
`precision_score(y_test, y_pred)`

Out[21]: 0.8611111111111112

In [22]: `recall_score(y_test, y_pred)`

Out[22]: 0.7560975609756098

In [23]: `from sklearn.metrics import f1_score`
`f1_score(y_test, y_pred)`

Out[23]: 0.8051948051948052

In []:

Experiment - 6

Data Analytics III

1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset

```
In [1]: import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [3]: iris= pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\6\Iris.csv")
```

```
In [4]: iris.head()
```

```
Out[4]:
```

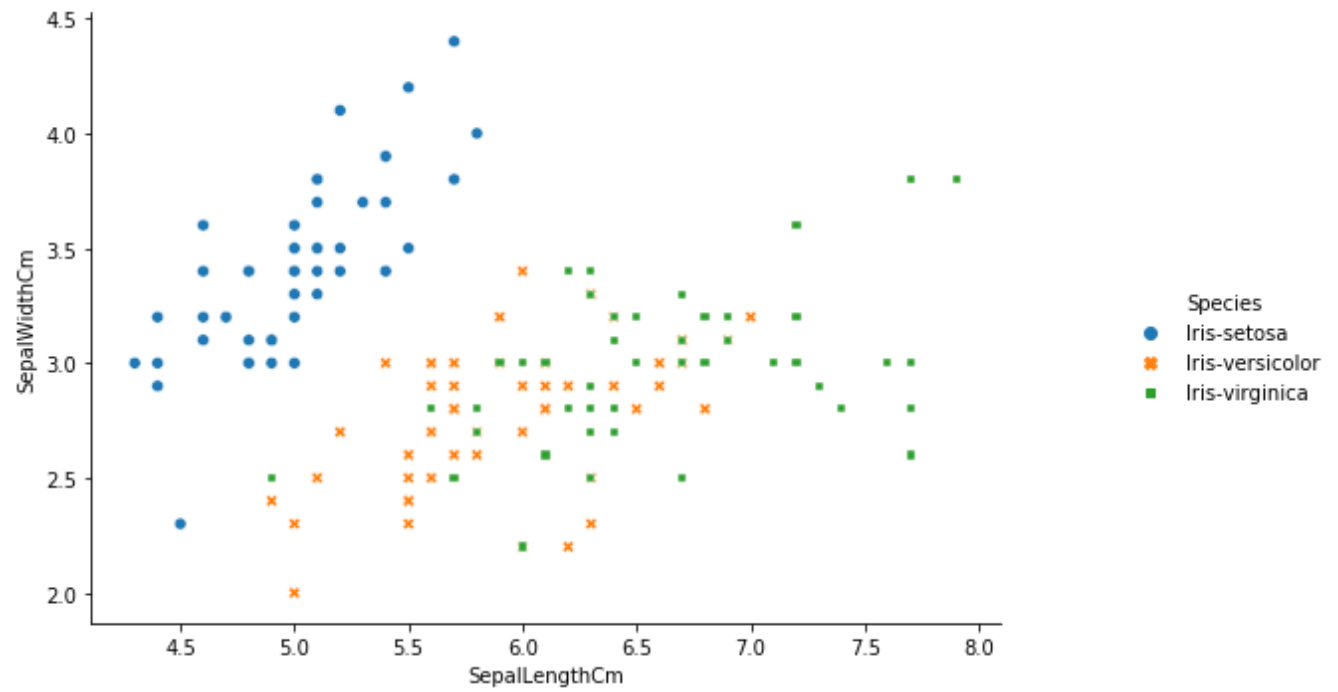
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [5]: iris['Species'].unique()
```

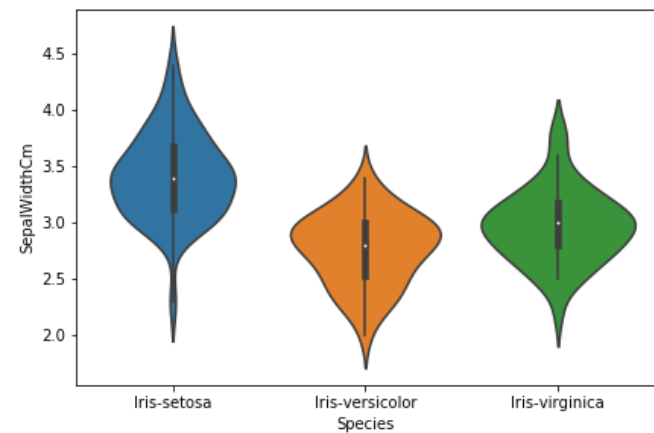
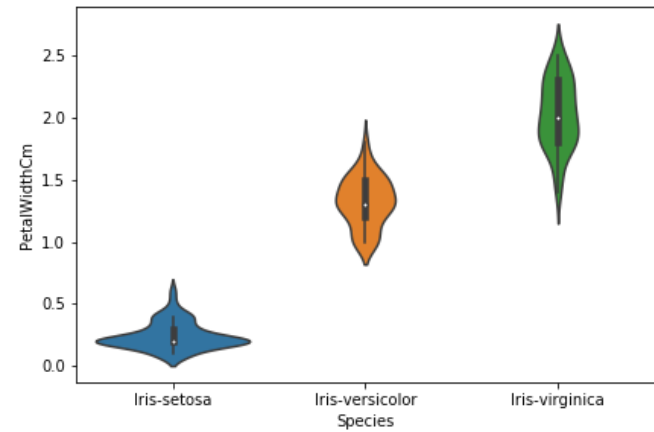
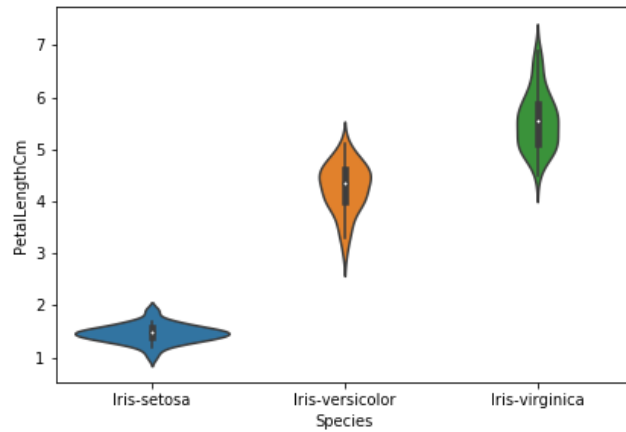
```
Out[5]: array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

```
In [6]: iris.drop(columns="Id",inplace=True)
```

```
In [7]: g=sns.relplot(x='SepalLengthCm',y='SepalWidthCm',data=iris,hue='Species',style='Species')
g.fig.set_size_inches(10,5)
plt.show()
```



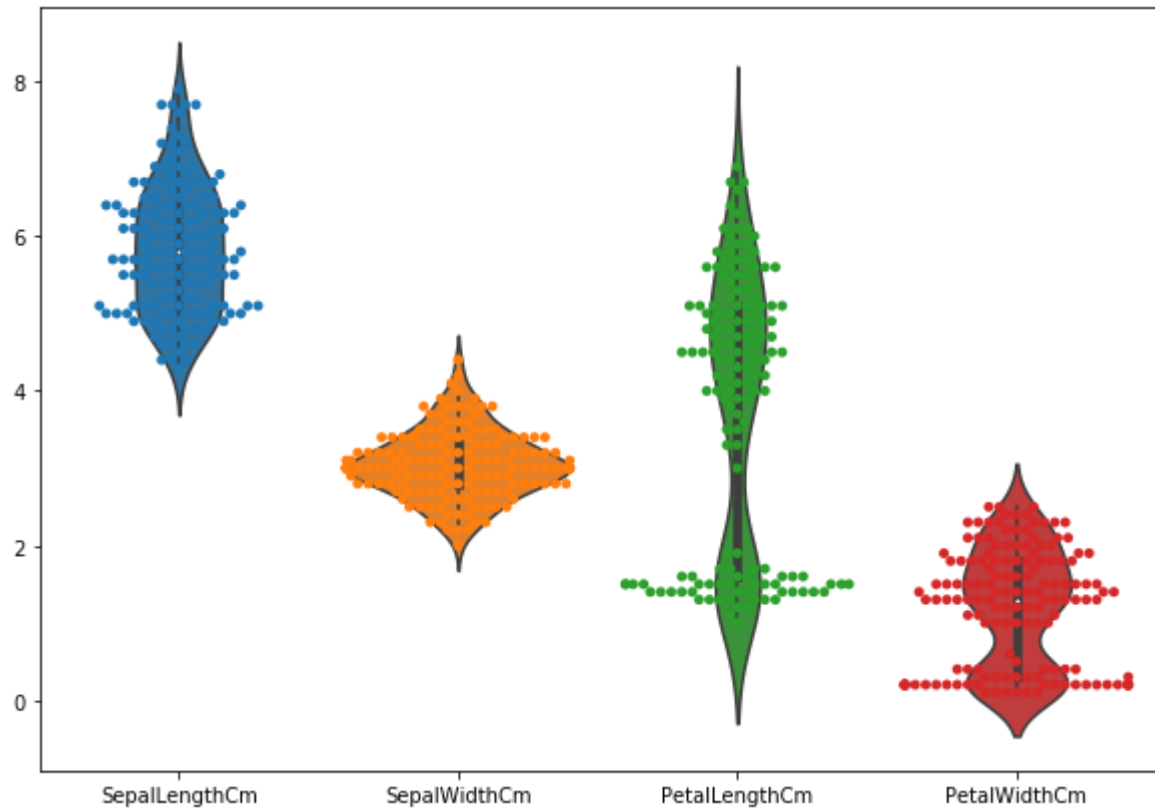
```
In [8]: plt.figure(figsize=(15,10))
plt.subplot(2,2,1)
sns.violinplot(x='Species',y='PetalLengthCm',data=iris)
plt.subplot(2,2,2)
sns.violinplot(x='Species',y='PetalWidthCm',data=iris)
plt.subplot(2,2,3)
sns.violinplot(x='Species',y='SepalLengthCm',data=iris)
plt.subplot(2,2,4)
sns.violinplot(x='Species',y='SepalWidthCm',data=iris)
plt.show()
```



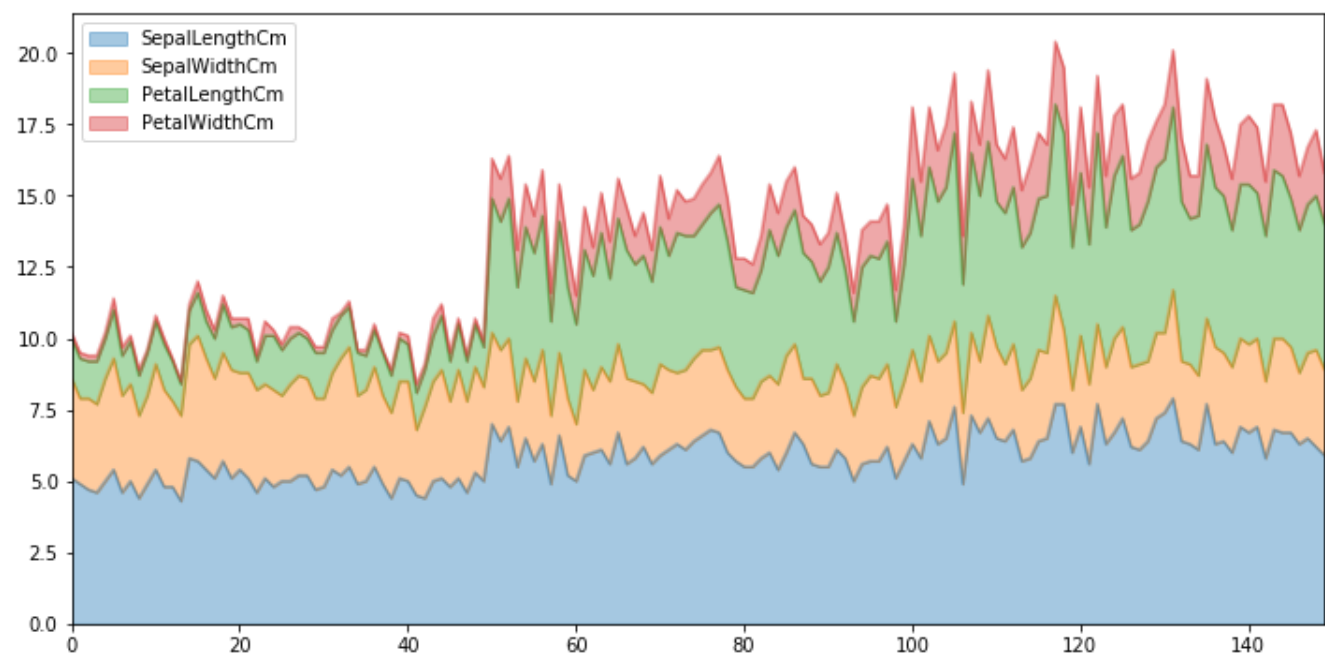
```
In [9]: plt.subplots(figsize=(10,7))
sns.violinplot(data=iris)
sns.swarmplot( data=iris)
plt.show()
```

C:\Users\HP\Anaconda3\lib\site-packages\seaborn\categorical.py:1296: UserWarning: 9.3% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.

```
warnings.warn(msg, UserWarning)
```



```
In [10]: iris.plot.area(y=[ 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'],alpha=
```

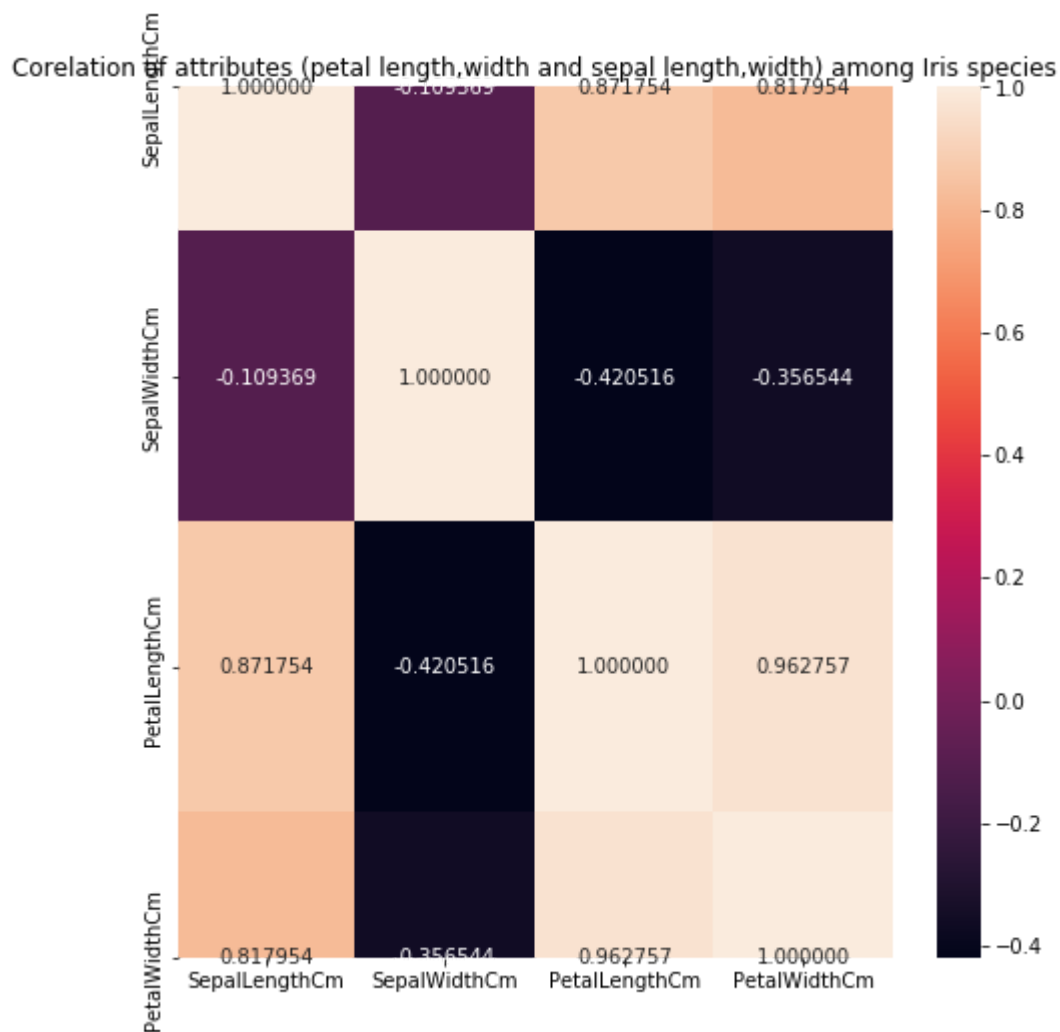


```
In [11]: iris.corr()
```

Out[11]:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
SepalLengthCm	1.000000	-0.109369	0.871754	0.817954
SepalWidthCm	-0.109369	1.000000	-0.420516	-0.356544
PetalLengthCm	0.871754	-0.420516	1.000000	0.962757
PetalWidthCm	0.817954	-0.356544	0.962757	1.000000

```
In [12]: plt.subplots(figsize = (8,8))
sns.heatmap(iris.corr(),annot=True,fmt="f").set_title("Corelation of attributes (petal
plt.show()
```



```
In [13]: X=iris.iloc[:,0:4].values
y=iris.iloc[:,4].values
```

```
In [14]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
```

```
In [18]: #Metrics
from sklearn.metrics import make_scorer, accuracy_score, precision_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

#Model Select
from sklearn.naive_bayes import GaussianNB
```

```
In [19]: #Train and Test split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=0)

-----
NameError                                Traceback (most recent call last)
<ipython-input-19-0918305e7a5c> in <module>
      1 #Train and Test split
----> 2 X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=
0)

NameError: name 'train_test_split' is not defined
```

```
In [44]: gaussian = GaussianNB()
gaussian.fit(X_train, y_train)
Y_pred = gaussian.predict(X_test)
accuracy_nb=round(accuracy_score(y_test,Y_pred)* 100, 2)
acc_gaussian = round(gaussian.score(X_train, y_train) * 100, 2)

cm = confusion_matrix(y_test, Y_pred)
accuracy = accuracy_score(y_test,Y_pred)
precision =precision_score(y_test, Y_pred,average='micro')
recall = recall_score(y_test, Y_pred,average='micro')
f1 = f1_score(y_test,Y_pred,average='micro')
print('Confusion matrix for Naive Bayes\n',cm)
print('accuracy_Naive Bayes: %.3f' %accuracy)
print('precision_Naive Bayes: %.3f' %precision)
print('recall_Naive Bayes: %.3f' %recall)
print('f1-score_Naive Bayes : %.3f' %f1)
```

```
Confusion matrix for Naive Bayes
[[16  0  0]
 [ 0 18  0]
 [ 0  0 11]]
accuracy_Naive Bayes: 1.000
precision_Naive Bayes: 1.000
recall_Naive Bayes: 1.000
f1-score_Naive Bayes : 1.000
```

In []:

In []:

Experiment - 7

Text Analytics

1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of document by calculating Term Frequency and Inverse Document Frequency.

```
In [1]: pip install PyPDF2
```

Collecting PyPDF2

Downloading <https://files.pythonhosted.org/packages/b4/01/68fcc0d43daf4c6bdbbc6b33cc3f77bda531c86b174cac56ef0ffdb96faab/PyPDF2-1.26.0.tar.gz> (77kB)

Building wheels for collected packages: PyPDF2

Building wheel for PyPDF2 (setup.py): started

Building wheel for PyPDF2 (setup.py): finished with status 'done'

Created wheel for PyPDF2: filename=PyPDF2-1.26.0-cp37-none-any.whl size=61091 sha256=0d4e38a2d84a2eb596926071afcfcc9af7ee84e4a96279e01fcc241d05e7915c

Stored in directory: C:\Users\HP\AppData\Local\pip\Cache\wheels\53\84\19\35bc977c8bf5f0c23a8a011aa958acd4da4bbd7a229315c1b7

Successfully built PyPDF2

Installing collected packages: PyPDF2

Successfully installed PyPDF2-1.26.0

Note: you may need to restart the kernel to use updated packages.

```
In [2]: pip install python-docx
```

Collecting python-docx

Downloading <https://files.pythonhosted.org/packages/8b/a0/52729ce4aa026f31b74cc877be1d11e4ddeaa361dc7aebec148171644b33/python-docx-0.8.11.tar.gz> (5.6MB)

Requirement already satisfied: lxml>=2.3.2 in c:\users\hp\anaconda3\lib\site-packages (from python-docx) (4.4.1)

Building wheels for collected packages: python-docx

Building wheel for python-docx (setup.py): started

Building wheel for python-docx (setup.py): finished with status 'done'

Created wheel for python-docx: filename=python_docx-0.8.11-cp37-none-any.whl size=184607 sha256=4e01ad30c11a4aa6b83f7f4ed6a7d696a2fb4d55f2dfe2cef0f0f8cae90d846e

Stored in directory: C:\Users\HP\AppData\Local\pip\Cache\wheels\56\90\51\7c7b70b38633ae04e7fb963b1c70f63fd6fc01c075b8230adc

Successfully built python-docx

Installing collected packages: python-docx

Successfully installed python-docx-0.8.11

Note: you may need to restart the kernel to use updated packages.

```
In [4]: # importing required modules
import PyPDF2

# creating a pdf file object
pdfFileObj = open(r"D:\College\TE\SEM-2\Practical\DSBDA\7\sample1.pdf", 'rb')

# creating a pdf reader object
pdfReader = PyPDF2.PdfFileReader(pdfFileObj)

# printing number of pages in pdf file
print(pdfReader.numPages)

# creating a page object
pageObj = pdfReader.getPage(0)

# extracting text from page
print(pageObj.extractText())

# closing the pdf file object
pdfFileObj.close()
```

```
1
Welcome to Smallpdf
Digital DocumentsŠAll In One Place
Access Files Anytime, Anywhere
Enhance Documents in One Click
Collaborate With Others
With the new Smallpdf experience, you can
freely upload, organize, and share digital
documents. When you enable the
,Storage™
option
```

your computer, phone, or tablet. We™ll also

Smallpdf Mobile App to our
online portal

you with an array of options to convert,
compress, or modify it.
Forget mundane administrative tasks. With
Smallpdf, you can request e-signatures, send

Smallpdf G Suite
App for your entire organization.
Ready to take document management to the next level?

```
In [9]: # import docx NOT python-docx
import docx

# create an instance of a word document
doc = docx.Document()

# add a heading of level 0 (largest heading)
doc.add_heading('Heading for the document', 0)

# add a paragraph and store
# the object in a variable
doc_para = doc.add_paragraph('Your paragraph goes here, ')

# add a run i.e, style like
# bold, italic, underline, etc.
doc_para.add_run('hey there, bold here').bold = True
doc_para.add_run(', and ')
doc_para.add_run('these words are italic').italic = True

# add a page break to start a new page
doc.add_page_break()

# add a heading of level 2
doc.add_heading('Heading level 2', 2)

# pictures can also be added to our word document
# width is optional
doc.add_picture(r"D:\College\TE\SEM-2\Practical\DSBDA\7\index.jpg")

# now save the document to a location
doc.save('new_doc')
```

```
In [10]: pip install nltk
```

Requirement already satisfied: nltk in c:\users\hp\anaconda3\lib\site-packages (3.4.5)
Requirement already satisfied: six in c:\users\hp\anaconda3\lib\site-packages (from nltk) (1.12.0)
Note: you may need to restart the kernel to use updated packages.

```
In [11]: import nltk
nltk.download()
nltk.download('punkt')
```

showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml (https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml)

```
[nltk_data] Downloading package punkt to
[nltk_data]      C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

```
Out[11]: True
```

In [12]: *#Sentence Tokenization*

```
sentence_data = "The First sentence is about Python. The Second: about Django. You can  
nltk_tokens = nltk.sent_tokenize(sentence_data)  
print (nltk_tokens)
```

```
['The First sentence is about Python.', 'The Second: about Django.', 'You can learn Py  
thon,Django and Data Ananlysis here.']
```

In [13]: *#Non English Language Tokenization*

```
german_tokenizer = nltk.data.load('tokenizers/punkt/german.pickle')  
german_tokens=german_tokenizer.tokenize('Wie geht es Ihnen? Gut, danke.')  
print(german_tokens)
```

```
['Wie geht es Ihnen?', 'Gut, danke.']
```

In [14]: *#Word Tokenization*

```
word_data = "It originated from the idea that there are readers who prefer learning new  
nltk_tokens = nltk.word_tokenize(word_data)  
print (nltk_tokens)
```

```
['It', 'originated', 'from', 'the', 'idea', 'that', 'there', 'are', 'readers', 'who',  
'prefer', 'learning', 'new', 'skills', 'from', 'the', 'comforts', 'of', 'their', 'draw  
ing', 'rooms']
```

In [15]: *#Word Tokenization*

```
from nltk.corpus import stopwords  
from nltk.tokenize import word_tokenize, sent_tokenize
```

#Dummy text

```
txt = "He is a boy. "\n      "She is a girl"
```

```
word_tokens = word_tokenize(txt)
```

```
print(word_tokens)
```

```
['He', 'is', 'a', 'boy', '.', 'She', 'is', 'a', 'girl']
```

In [16]: *#Part of Speech (POS) tagging*

```
import nltk
nltk.download('averaged_perceptron_tagger')
from nltk.tokenize import word_tokenize
text = word_tokenize("Hello welcome to the world of to learn Categorizing and POS Tagging")
nltk.pos_tag(text)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
```

Out[16]:

```
[('Hello', 'NNP'),
 ('welcome', 'NN'),
 ('to', 'TO'),
 ('the', 'DT'),
 ('world', 'NN'),
 ('of', 'IN'),
 ('to', 'TO'),
 ('learn', 'VB'),
 ('Categorizing', 'NNP'),
 ('and', 'CC'),
 ('POS', 'NNP'),
 ('Tagging', 'NNP'),
 ('with', 'IN'),
 ('NLTK', 'NNP'),
 ('and', 'CC'),
 ('Python', 'NNP')]
```

In [17]: `import nltk`
`nltk.download('stopwords')`
`nltk.download('averaged_perceptron_tagger')`

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
```

Out[17]: True

```
In [18]: from nltk.corpus import stopwords
print(stopwords.words('english'))
```

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn', "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]
```

```
In [19]: #Stopwords removal from sentence
```

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

example_sent = """This is a sample sentence,
                  showing off the stop words filtration."""

stop_words = set(stopwords.words('english'))

word_tokens = word_tokenize(example_sent)

filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]

filtered_sentence = []

for w in word_tokens:
    if w not in stop_words:
        filtered_sentence.append(w)

print("Tokenized:", word_tokens)
print("Stop Words Removed:", filtered_sentence)
```

```
Tokenized: ['This', 'is', 'a', 'sample', 'sentence', ',', 'showing', 'off', 'the', 'stop', 'words', 'filtration', '.']
Stop Words Removed: ['This', 'sample', 'sentence', ',', 'showing', 'stop', 'words', 'filtration', '.']
```

```
In [21]: #Stopwords from input file

import io
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# word_tokenize accepts
# a string as an input, not a file.
stop_words = set(stopwords.words('english'))
file1 = open(r"D:\College\TE\SEM-2\Practical\DSBDA\7\text.txt")

# Use this to read file content as a stream:
line = file1.read()
words = line.split()
for r in words:
    if not r in stop_words:
        appendFile = open('filteredtext.txt','a')
        appendFile.write(" "+r)
        appendFile.close()
```

In [22]: *#Stemming*

```
import nltk
from nltk.stem.porter import PorterStemmer
porter_stemmer = PorterStemmer()

word_data = "It vijaying meeting better vijayed vijays eats skills originated from the"
# First Word tokenization
nltk_tokens = nltk.word_tokenize(word_data)
#Next find the roots of the word
for w in nltk_tokens:
    print("Actual: %s Stem: %s" % (w,porter_stemmer.stem(w)))
```

```
Actual: It Stem: It
Actual: vijaying Stem: vijay
Actual: meeting Stem: meet
Actual: better Stem: better
Actual: vijayed Stem: vijay
Actual: vijays Stem: vijay
Actual: eats Stem: eat
Actual: skills Stem: skill
Actual: originated Stem: origin
Actual: from Stem: from
Actual: the Stem: the
Actual: idea Stem: idea
Actual: that Stem: that
Actual: there Stem: there
Actual: are Stem: are
Actual: readers Stem: reader
Actual: who Stem: who
Actual: prefer Stem: prefer
Actual: learning Stem: learn
Actual: new Stem: new
Actual: skills Stem: skill
Actual: from Stem: from
Actual: the Stem: the
Actual: comforts Stem: comfort
Actual: of Stem: of
Actual: their Stem: their
Actual: drawing Stem: draw
Actual: rooms Stem: room
```

In [23]:

#Lemmatization

```
import nltk
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()

word_data = "It studies densely is better meeting studying vijaying vijayed vijays ski
nltk_tokens = nltk.word_tokenize(word_data)
for w in nltk_tokens:
    print("Actual: %s Lemma: %s" % (w,wordnet_lemmatizer.lemmatize(w)))
```

```
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```
Actual: It Lemma: It
Actual: studies Lemma: study
Actual: densely Lemma: densely
Actual: is Lemma: is
Actual: better Lemma: better
Actual: meeting Lemma: meeting
Actual: studying Lemma: studying
Actual: vijaying Lemma: vijaying
Actual: vijayed Lemma: vijayed
Actual: vijays Lemma: vijays
Actual: skills Lemma: skill
Actual: originated Lemma: originated
Actual: from Lemma: from
Actual: the Lemma: the
Actual: idea Lemma: idea
Actual: that Lemma: that
Actual: there Lemma: there
Actual: are Lemma: are
Actual: readers Lemma: reader
Actual: who Lemma: who
Actual: prefer Lemma: prefer
Actual: learning Lemma: learning
Actual: new Lemma: new
Actual: skills Lemma: skill
Actual: from Lemma: from
Actual: the Lemma: the
Actual: comforts Lemma: comfort
Actual: of Lemma: of
Actual: their Lemma: their
Actual: drawing Lemma: drawing
Actual: rooms Lemma: room
```

In [24]:

#Expt.No.7 2nd Operation

```
import pandas as pd
import sklearn as sk
import math
```

```
In [25]: first_sentence = "Data Science is the best job of the 21st century"
second_sentence = "Machine learning is the key for data science"

#split so each word have their own string
first_sentence = first_sentence.split(" ")
second_sentence = second_sentence.split(" ")#join them to remove common duplicate words
total= set(first_sentence).union(set(second_sentence))
print(total)
```

```
{'century', 'Data', 'of', 'for', '21st', 'job', 'learning', 'the', 'Science', 'is', 'key', 'Machine', 'science', 'best', 'data'}
```

```
In [26]: #count the words

wordDictA = dict.fromkeys(total, 0)
wordDictB = dict.fromkeys(total, 0)
for word in first_sentence:
    wordDictA[word]+=1

for word in second_sentence:
    wordDictB[word]+=1

pd.DataFrame([wordDictA, wordDictB])
```

Out[26]:

	century	Data	of	for	21st	job	learning	the	Science	is	key	Machine	science	best	data
0	1	1	1	0	1	1	0	2	1	1	0	0	0	1	0
1	0	0	0	1	0	0	1	1	0	1	1	1	1	0	1

```
In [27]: #Compute Term Frequency(TF)

def computeTF(wordDict, doc):
    tfDict = {}
    corpusCount = len(doc)
    for word, count in wordDict.items():
        tfDict[word] = count/float(corpusCount)
    return(tfDict)

#running our sentences through the tf function:
tfFirst = computeTF(wordDictA, first_sentence)
tfSecond = computeTF(wordDictB, second_sentence)

#Converting to dataframe for visualization
pd.DataFrame([tfFirst, tfSecond])
```

Out[27]:

	century	Data	of	for	21st	job	learning	the	Science	is	key	Machine	science	best	d
0	0.1	0.1	0.1	0.000	0.1	0.1	0.000	0.200	0.1	0.100	0.000	0.000	0.000	0.1	0.0
1	0.0	0.0	0.0	0.125	0.0	0.0	0.125	0.125	0.0	0.125	0.125	0.125	0.125	0.0	0.0

```
In [28]: #Compute Inverse Document Frequency(IDF)

def computeIDF(docList):
    idfDict = {}
    N = len(docList)

    idfDict = dict.fromkeys(docList[0].keys(), 0)
    for word, val in idfDict.items():
        idfDict[word] = math.log10(N / (float(val) + 1))

    return(idfDict)

#inputing our sentences in the log file
idfs = computeIDF([wordDictA, wordDictB])
```

```
In [29]: #Compute Term Frequency(TF) - Inverse Document Frequency(IDF)

def computeTFIDF(tfBow, idfs):
    tfidf = {}
    for word, val in tfBow.items():
        tfidf[word] = val*idfs[word]
    return(tfidf)

#running our two sentences through the IDF:
idfFirst = computeTFIDF(tfFirst, idfs)
idfSecond = computeTFIDF(tfSecond, idfs)

#putting it in a dataframe
pd.DataFrame([idfFirst, idfSecond])
```

Out[29]:

	century	Data	of	for	21st	job	learning	the	Science	is
0	0.030103	0.030103	0.030103	0.000000	0.030103	0.030103	0.000000	0.060206	0.030103	0.030103
1	0.000000	0.000000	0.000000	0.037629	0.000000	0.000000	0.037629	0.037629	0.000000	0.037629

```
In [30]: #Compute TF-IDF

#first step is to import the library
from sklearn.feature_extraction.text import TfidfVectorizer

#for the sentence, make sure all words are lowercase or you will run #into error. for s
firstV= "Data Science is the sexiest job of the 21st century"
secondV= "machine learning is the key for data science"

#calling the TfidfVectorizer
vectorize= TfidfVectorizer()

#fitting the model and passing our sentences right away:
response= vectorize.fit_transform([firstV, secondV])

print(response)
```

```
(0, 1)      0.34211869506421816
(0, 0)      0.34211869506421816
(0, 9)      0.34211869506421816
(0, 5)      0.34211869506421816
(0, 11)     0.34211869506421816
(0, 12)     0.48684053853849035
(0, 4)      0.24342026926924518
(0, 10)     0.24342026926924518
(0, 2)      0.24342026926924518
(1, 3)      0.40740123733358447
(1, 6)      0.40740123733358447
(1, 7)      0.40740123733358447
(1, 8)      0.40740123733358447
(1, 12)     0.28986933576883284
(1, 4)      0.28986933576883284
(1, 10)     0.28986933576883284
(1, 2)      0.28986933576883284
```

In []:

Experiment - 8

Data Visualization I

1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

In [1]: `pip install seaborn`

```
Requirement already satisfied: seaborn in c:\users\hp\anaconda3\lib\site-packages (0.9.0)
Requirement already satisfied: matplotlib>=1.4.3 in c:\users\hp\anaconda3\lib\site-packages (from seaborn) (3.1.1)
Requirement already satisfied: numpy>=1.9.3 in c:\users\hp\anaconda3\lib\site-packages (from seaborn) (1.16.5)
Requirement already satisfied: scipy>=0.14.0 in c:\users\hp\anaconda3\lib\site-packages (from seaborn) (1.3.1)
Requirement already satisfied: pandas>=0.15.2 in c:\users\hp\anaconda3\lib\site-packages (from seaborn) (0.25.1)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib>=1.4.3->seaborn) (0.10.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib>=1.4.3->seaborn) (1.1.0)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib>=1.4.3->seaborn) (2.4.2)
Requirement already satisfied: python-dateutil>=2.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib>=1.4.3->seaborn) (2.8.0)
Requirement already satisfied: pytz>=2017.2 in c:\users\hp\anaconda3\lib\site-packages (from pandas>=0.15.2->seaborn) (2019.3)
Requirement already satisfied: six in c:\users\hp\anaconda3\lib\site-packages (from cycler>=0.10->matplotlib>=1.4.3->seaborn) (1.12.0)
Requirement already satisfied: setuptools in c:\users\hp\anaconda3\lib\site-packages (from kiwisolver>=1.0.1->matplotlib>=1.4.3->seaborn) (41.4.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

dataset = sns.load_dataset('titanic')

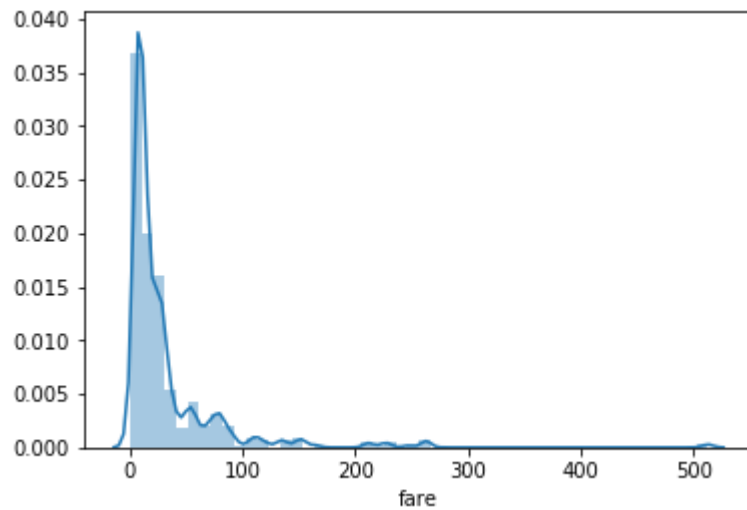
dataset.head()
```

Out[2]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embarked
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Sou
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Sou
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Sou
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sou

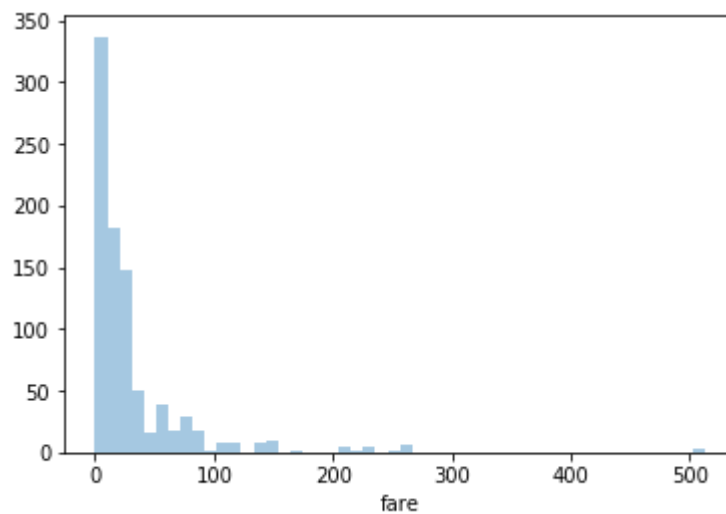
```
In [3]: sns.distplot(dataset['fare'])
```

```
Out[3]: <matplotlib.axes._subplots.AxesSubplot at 0x2367329efc8>
```



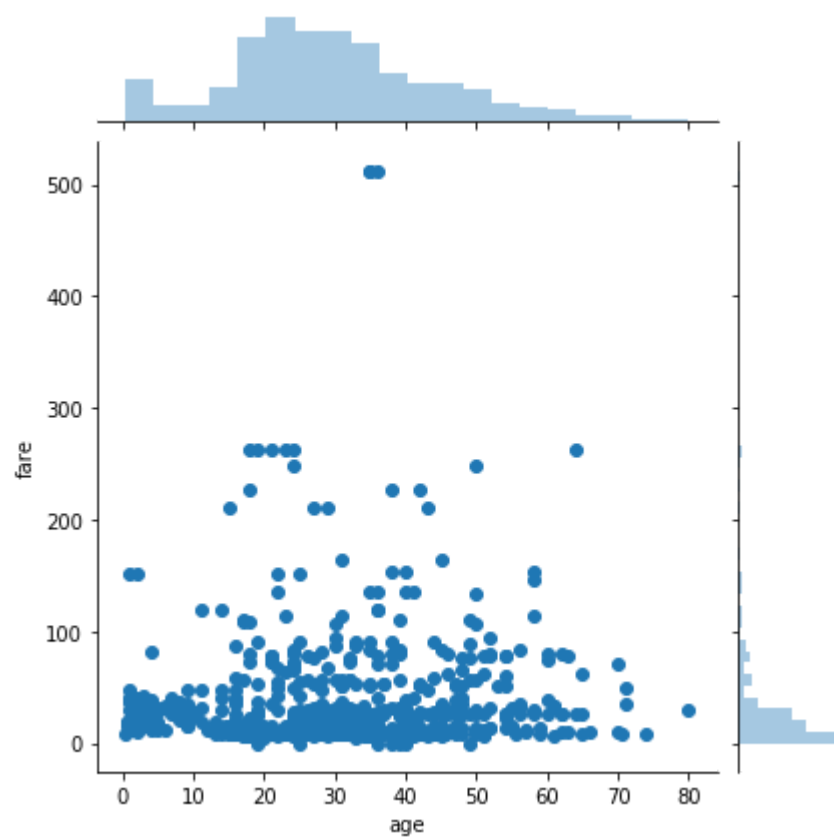
```
In [4]: sns.distplot(dataset['fare'], kde=False)
```

```
Out[4]: <matplotlib.axes._subplots.AxesSubplot at 0x236736415c8>
```



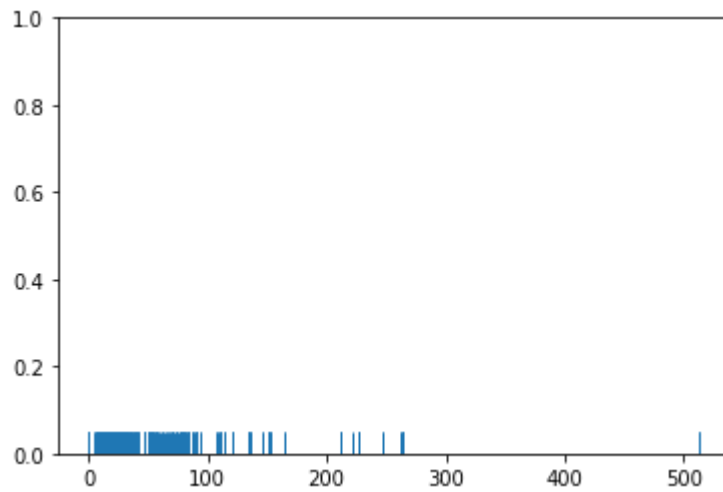
```
In [5]: sns.jointplot(x='age', y='fare', data=dataset)
```

```
Out[5]: <seaborn.axisgrid.JointGrid at 0x23673741888>
```



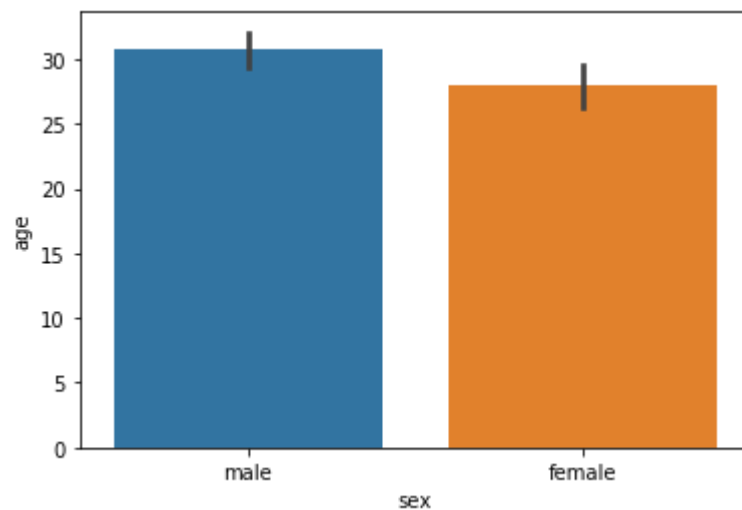
```
In [6]: sns.rugplot(dataset['fare'])
```

```
Out[6]: <matplotlib.axes._subplots.AxesSubplot at 0x23673972c08>
```



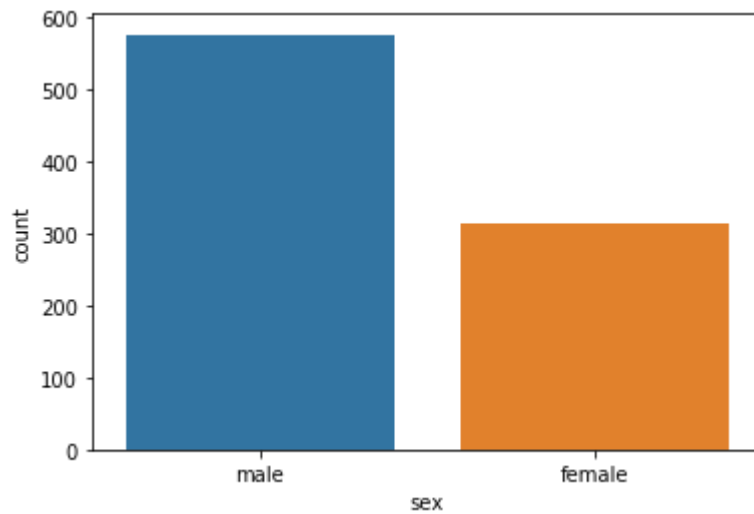
```
In [7]: sns.barplot(x='sex', y='age', data=dataset)
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0x23673a2f648>
```



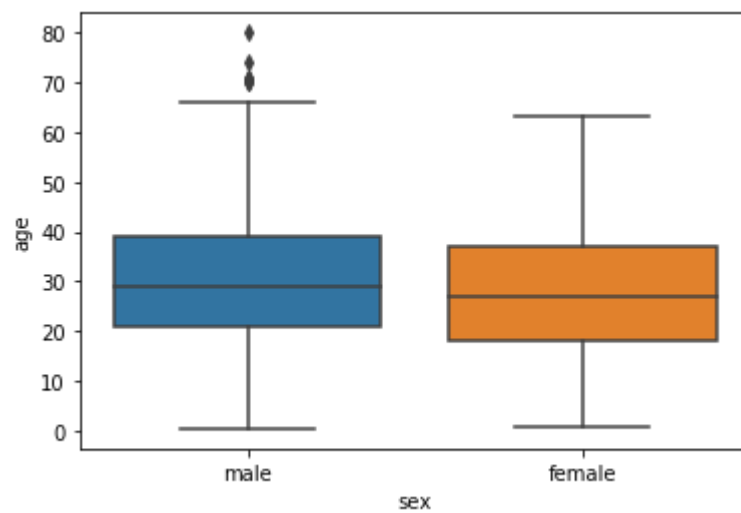
```
In [8]: sns.countplot(x='sex', data=dataset)
```

```
Out[8]: <matplotlib.axes._subplots.AxesSubplot at 0x2366d724a48>
```



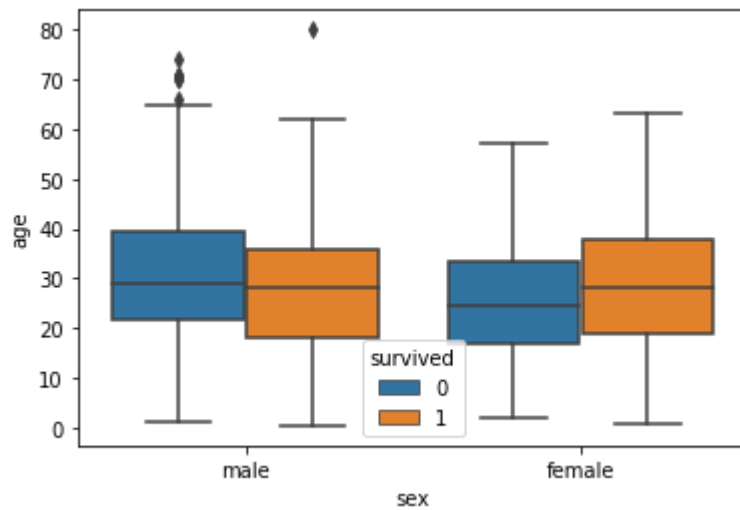
```
In [9]: sns.boxplot(x='sex', y='age', data=dataset)
```

```
Out[9]: <matplotlib.axes._subplots.AxesSubplot at 0x23673afc208>
```



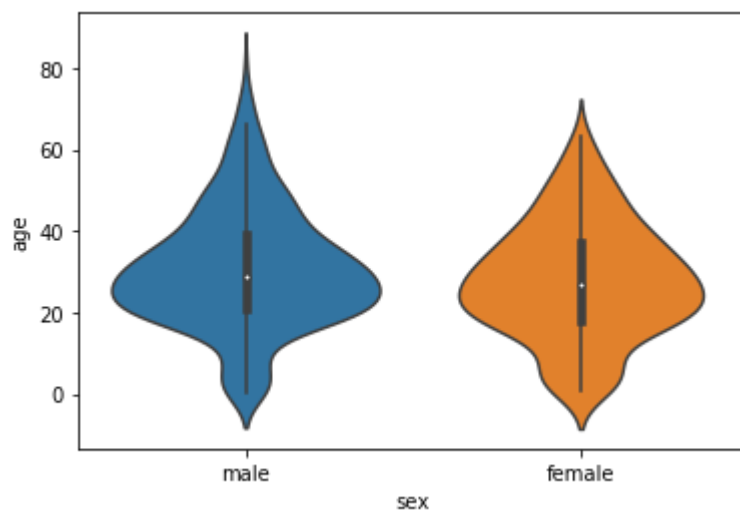
```
In [10]: sns.boxplot(x='sex', y='age', data=dataset, hue="survived")
```

```
Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0x23673b80508>
```



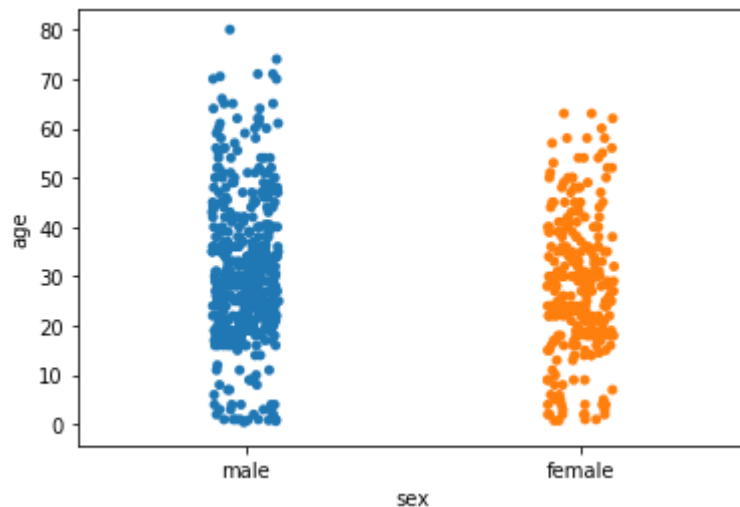
```
In [11]: sns.violinplot(x='sex', y='age', data=dataset)
```

```
Out[11]: <matplotlib.axes._subplots.AxesSubplot at 0x23673c46e48>
```



```
In [12]: sns.stripplot(x='sex', y='age', data=dataset)
```

```
Out[12]: <matplotlib.axes._subplots.AxesSubplot at 0x23673ca1888>
```

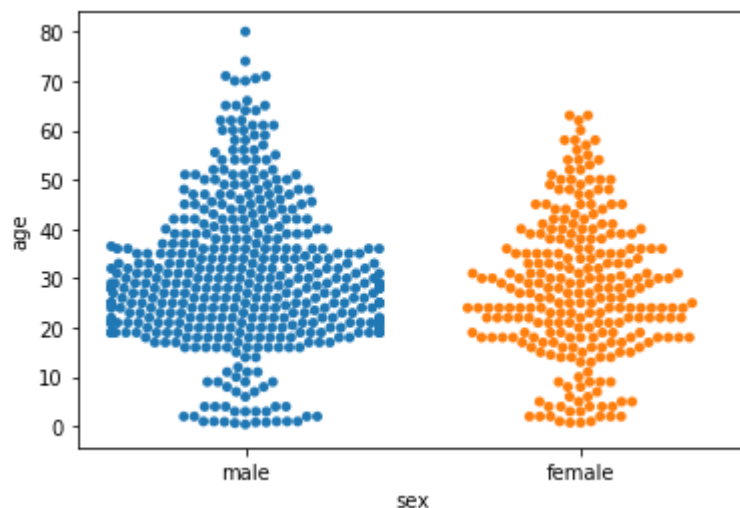


```
In [13]: sns.swarmplot(x='sex', y='age', data=dataset)
```

C:\Users\HP\Anaconda3\lib\site-packages\seaborn\categorical.py:1324: RuntimeWarning: invalid value encountered in less
off_low = points < low_gutter

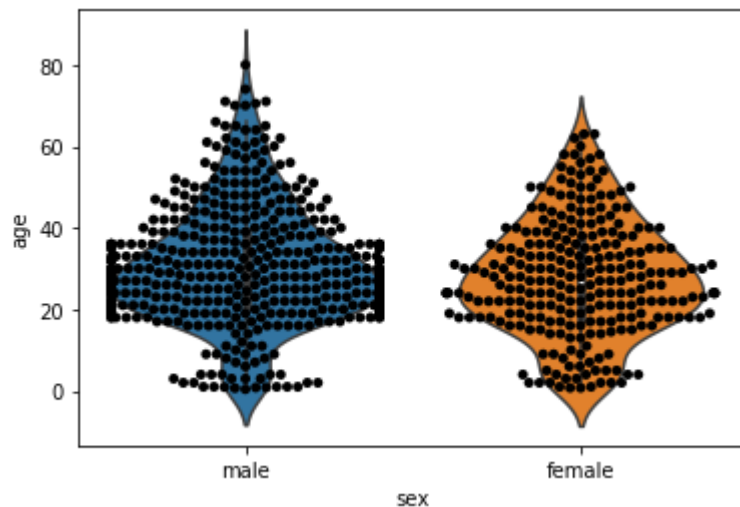
C:\Users\HP\Anaconda3\lib\site-packages\seaborn\categorical.py:1328: RuntimeWarning: invalid value encountered in greater
off_high = points > high_gutter

```
Out[13]: <matplotlib.axes._subplots.AxesSubplot at 0x23673d17488>
```



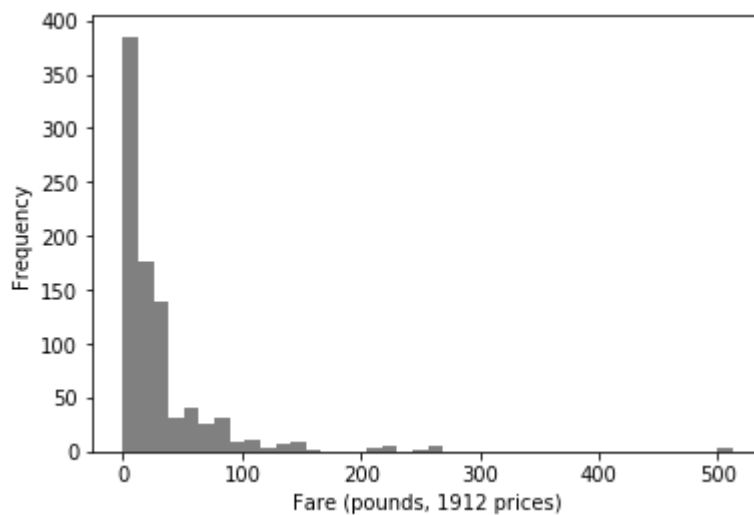
```
In [14]: sns.violinplot(x='sex', y='age', data=dataset)
sns.swarmplot(x='sex', y='age', data=dataset, color='black')
```

Out[14]: <matplotlib.axes._subplots.AxesSubplot at 0x23673dc5e48>



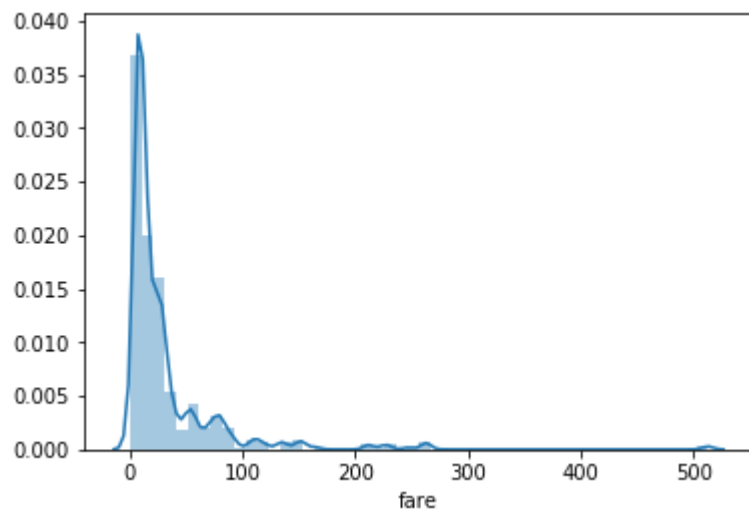
```
In [15]: #Expt. No. 8 Part-2
# histogram of fare
titanic_hist = dataset.fare.plot.hist(bins = 40, color = 'grey')
plt.xlabel('Fare (pounds, 1912 prices)')

plt.show(titanic_hist)
```



```
In [16]: sns.distplot(dataset['fare'])
```

```
Out[16]: <matplotlib.axes._subplots.AxesSubplot at 0x23673c28d48>
```



```
In [ ]:
```

Experiment - 9

Data Visualization II

1. Use the inbuilt dataset 'titanic' as used in the above problem. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age')
2. Write observations on the inference from the above statistics.

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

dataset = sns.load_dataset('titanic')

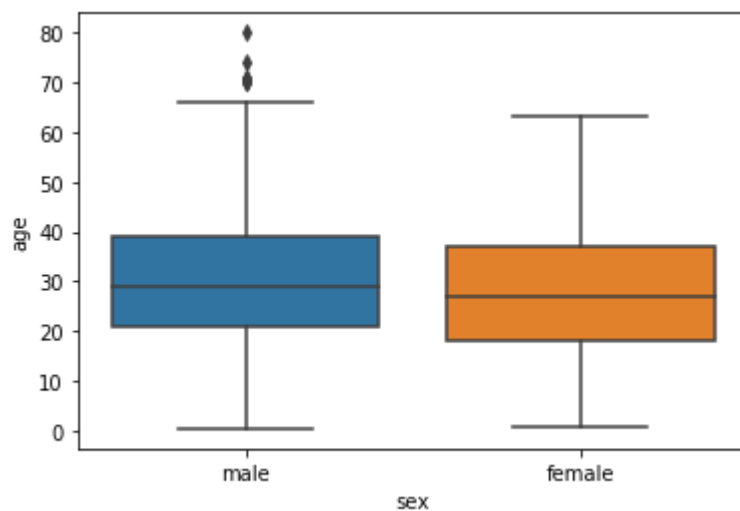
dataset.head()
```

Out[1]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embarked
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Sou
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Sou
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Sou
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sou

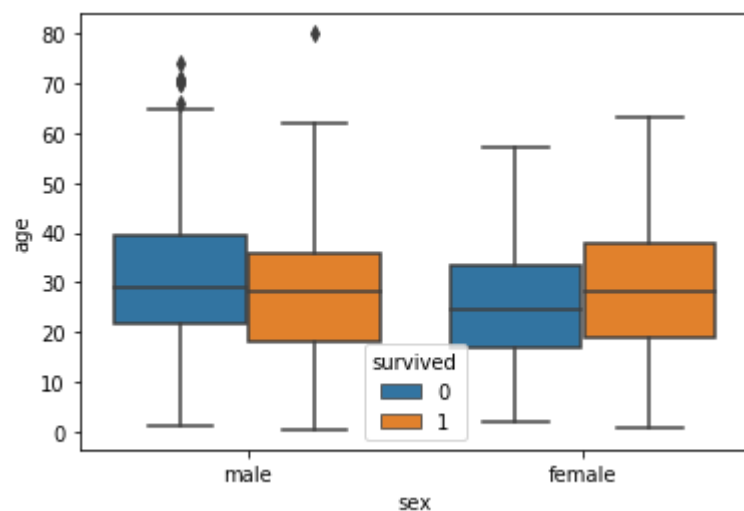
```
In [2]: sns.boxplot(x='sex', y='age', data=dataset)
```

Out[2]: <matplotlib.axes._subplots.AxesSubplot at 0x1527f09c1c8>



```
In [3]: sns.boxplot(x='sex', y='age', data=dataset, hue="survived")
```

```
Out[3]: <matplotlib.axes._subplots.AxesSubplot at 0x1527f3dcec8>
```



```
In [ ]:
```

Experiment - 10

Data Visualization III

Download the Iris flower dataset or any other dataset into a DataFrame. (e.g., <https://archive.ics.uci.edu/ml/datasets/Iris>). Scan the dataset and give the inference as:

1. List down the features and their types (e.g., numeric, nominal) available in the dataset.
2. Create a histogram for each feature in the dataset to illustrate the feature distributions.
3. Create a box plot for each feature in the dataset.
4. Compare distributions and identify outliers.

```
In [1]: import pandas as pd
import seaborn as sns
```

```
In [2]: data=pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\10\Iris.csv")
```

```
In [3]: data.head()
```

Out[3]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [4]: data.describe()
```

Out[4]:

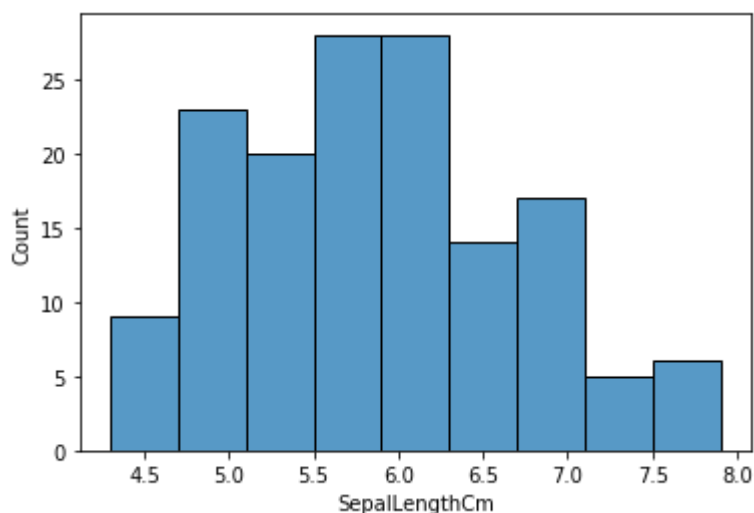
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
In [5]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 6 columns):  
Id                150 non-null int64  
SepalLengthCm     150 non-null float64  
SepalWidthCm      150 non-null float64  
PetalLengthCm     150 non-null float64  
PetalWidthCm      150 non-null float64  
Species           150 non-null object  
dtypes: float64(4), int64(1), object(1)  
memory usage: 7.2+ KB
```

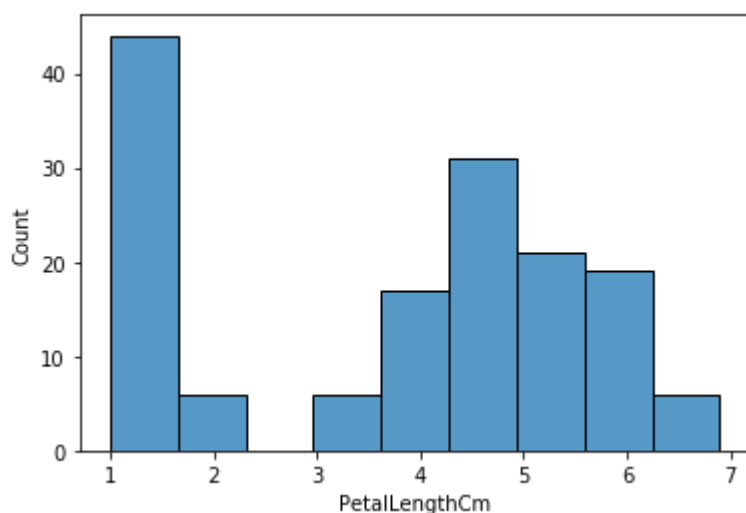
```
In [6]: sns.histplot(data=data, x="SepalLengthCm")
```

```
Out[6]: <matplotlib.axes._subplots.AxesSubplot at 0x2274e737808>
```



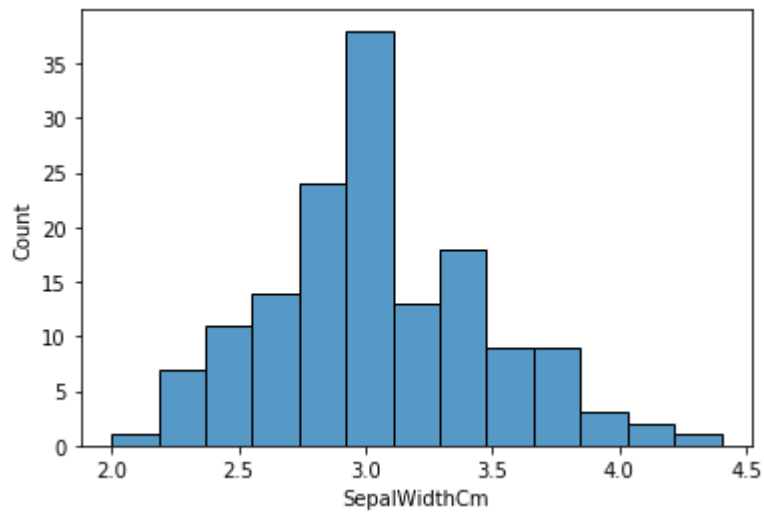
```
In [7]: sns.histplot(data=data, x="PetalLengthCm")
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0x2274eab8888>
```



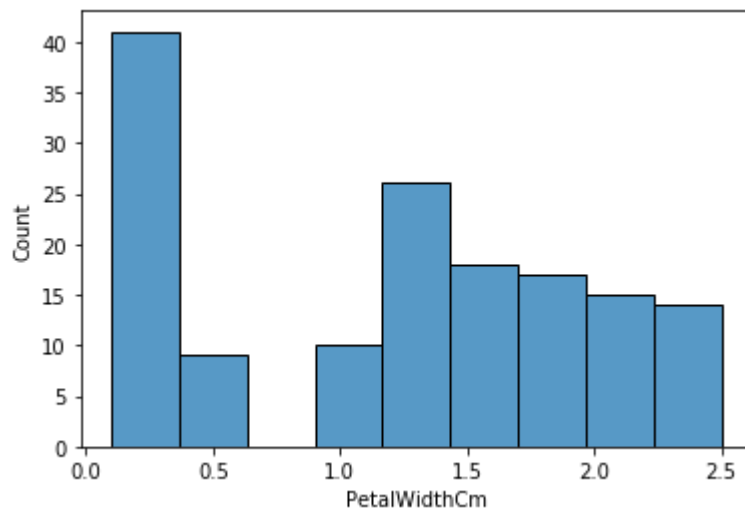
```
In [8]: sns.histplot(data=data, x="SepalWidthCm")
```

```
Out[8]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fb0c148>
```



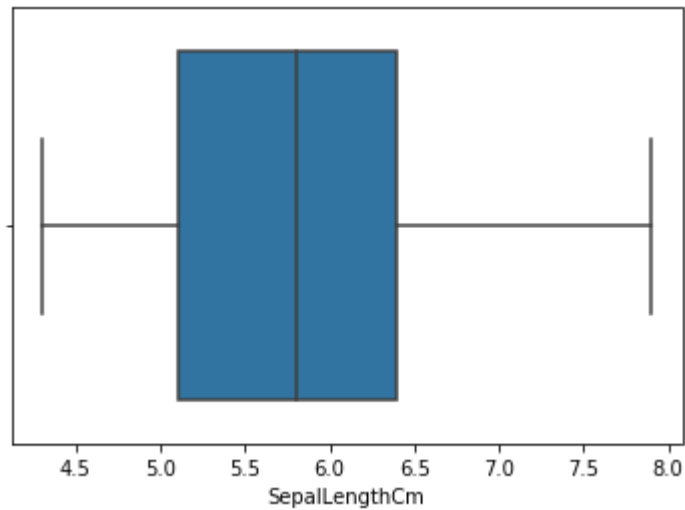
```
In [9]: sns.histplot(data=data, x="PetalWidthCm")
```

```
Out[9]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fbb9088>
```



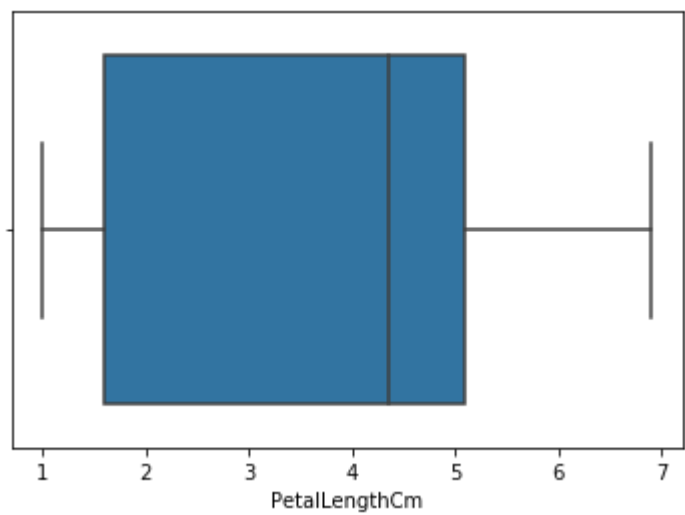
```
In [10]: sns.boxplot(x="SepalLengthCm", data=data)
```

```
Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fc47d88>
```



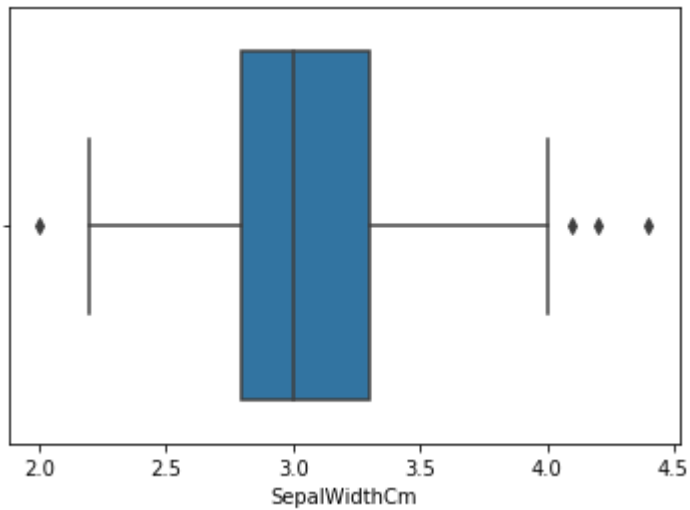
```
In [11]: sns.boxplot(x="PetalLengthCm", data=data)
```

```
Out[11]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fc977c8>
```



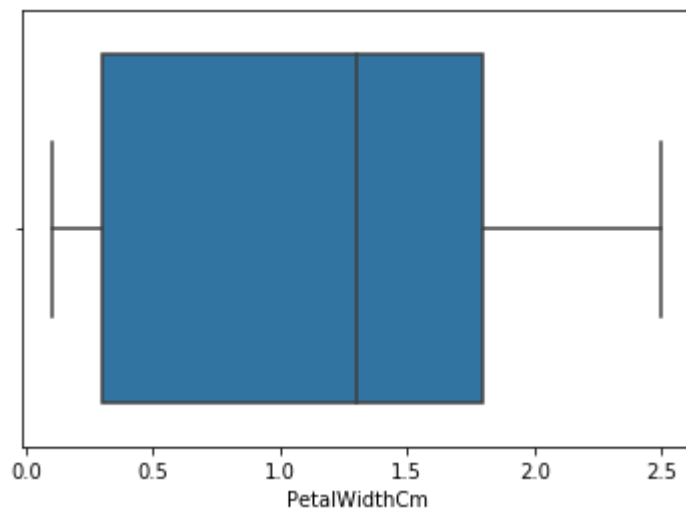
```
In [12]: sns.boxplot(x="SepalWidthCm", data=data)
```

```
Out[12]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fd2a3c8>
```



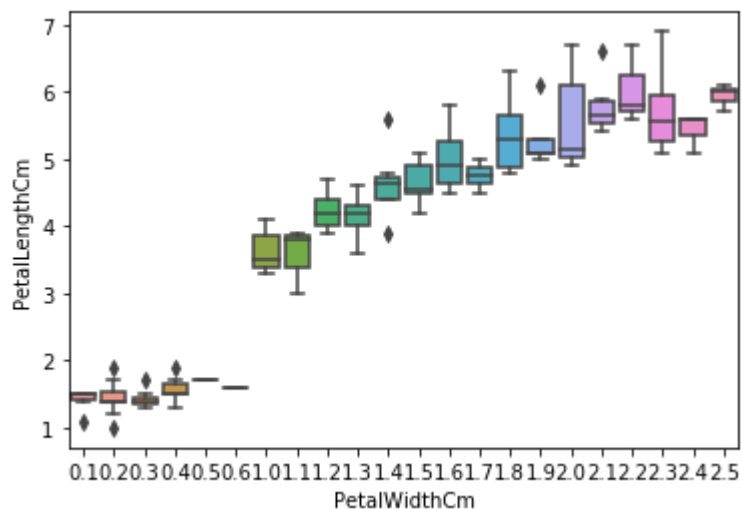
```
In [13]: sns.boxplot(x="PetalWidthCm", data=data)
```

```
Out[13]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fc58548>
```



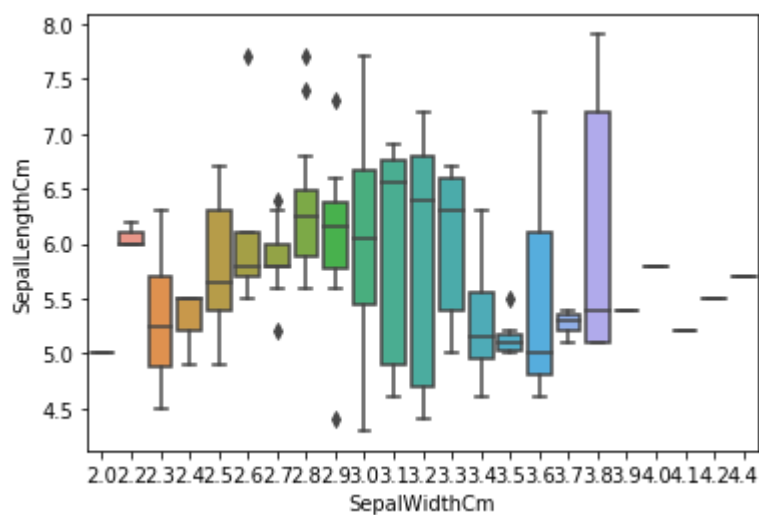
```
In [14]: sns.boxplot(x="PetalWidthCm", y="PetalLengthCm", data=data)
```

```
Out[14]: <matplotlib.axes._subplots.AxesSubplot at 0x2274fdd4648>
```



```
In [15]: sns.boxplot(x="SepalWidthCm", y="SepalLengthCm", data=data)
```

```
Out[15]: <matplotlib.axes._subplots.AxesSubplot at 0x2275004c188>
```



```
In [ ]:
```


Mini Project -1

Develop a movie recommendation model using the scikit-learn library in python.
Refer dataset

https://github.com/rashida048/Some-NLP-Projects/blob/master/movie_dataset.csv

```
In [1]: from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
In [3]: df = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\MP1\movie_dataset.csv")
```

```
In [4]: df.head()
```

Out[4]:

	index	budget	genres	homepage	id	keywords	original_language
0	0	237000000	Action Adventure Fantasy Science Fiction	http://www.avatarmovie.com/	19995	culture clash future space war space colony so...	
1	1	300000000	Adventure Fantasy Action	http://disney.go.com/disneypictures/pirates/	285	ocean drug abuse exotic island east india trad...	
2	2	245000000	Action Adventure Crime	http://www.sonypictures.com/movies/spectre/	206647	spy based on novel secret agent sequel mi6	
3	3	250000000	Action Crime Drama Thriller	http://www.thedarkknighttrises.com/	49026	dc comics crime fighter terrorist secret ident...	
4	4	260000000	Action Adventure Science Fiction	http://movies.disney.com/john-carter	49529	based on novel mars medallion space travel pri...	

5 rows × 24 columns

```
In [5]: features = ['keywords', 'cast', 'genres', 'director']
```

```
In [6]: def combine_features(row):
return row['keywords']+" "+row['cast']+" "+row['genres']+" "+row['director']
```

```
In [7]: for feature in features:
        df[feature] = df[feature].fillna('')

        df["combined_features"] = df.apply(combine_features,axis=1)
```

```
In [8]: cv = CountVectorizer()
        count_matrix = cv.fit_transform(df["combined_features"])
```

```
In [9]: cosine_sim = cosine_similarity(count_matrix)
```

```
In [10]: def get_title_from_index(index):
          return df[df.index == index]["title"].values[0]
          def get_index_from_title(title):
              return df[df.title == title]["index"].values[0]
```

```
In [11]: movie_user_likes = "Avatar"
          movie_index = get_index_from_title(movie_user_likes)
          similar_movies = list(enumerate(cosine_sim[movie_index]))
```

```
In [12]: sorted_similar_movies = sorted(similar_movies,key=lambda x:x[1],reverse=True)[1:]
```

```
In [13]: i=0
          print("Top 5 similar movies to "+movie_user_likes+" are:\n")
          for element in sorted_similar_movies:
              print(get_title_from_index(element[0]))
              i=i+1
              if i>5:
                  break
```

Top 5 similar movies to Avatar are:

Guardians of the Galaxy
Aliens
Star Wars: Clone Wars: Volume 1
Star Trek Into Darkness
Star Trek Beyond
Alien

Mini Project - 2

Use the following covid_vaccine_statewise.csv dataset and perform following analytics on the given dataset

[https://www.kaggle.com/sudalairajkumar/covid19-in-india?](https://www.kaggle.com/sudalairajkumar/covid19-in-india?select=covid_vaccine_statewise.csv)

[select=covid_vaccine_statewise.csv](#)

- Describe the dataset
- Number of persons state wise vaccinated for first dose in India
- Number of persons state wise vaccinated for second dose in India
- Number of Males vaccinated
- Number of females vaccinated

```
In [38]: import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
```

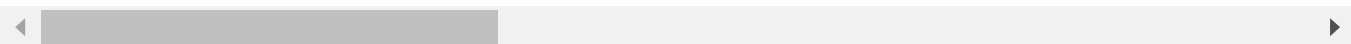
```
In [58]: vaccine_data = pd.read_csv(r"D:\College\TE\SEM-2\Practical\DSBDA\MP2\covid_vaccine_stat
```

```
In [59]: vaccine_data.head(5)
```

Out[59]:

	Updated On	State	Total Doses Administered	Sessions	Sites	First Dose Administered	Second Dose Administered	Male (Doses Administered)	Female (Doses Administered)
0	16/01/2021	India	48276.0	3455.0	2957.0	48276.0	0.0	NaN	NaN
1	17/01/2021	India	58604.0	8532.0	4954.0	58604.0	0.0	NaN	NaN
2	18/01/2021	India	99449.0	13611.0	6583.0	99449.0	0.0	NaN	NaN
3	19/01/2021	India	195525.0	17855.0	7951.0	195525.0	0.0	NaN	NaN
4	20/01/2021	India	251280.0	25472.0	10504.0	251280.0	0.0	NaN	NaN

5 rows × 10 columns



In [60]: vaccine_data.describe()

Out[60]:

	Total Doses Administered	Sessions	Sites	First Dose Administered	Second Dose Administered	Male (Doses Administered)	Female (Doses Administered)
count	7.621000e+03	7.621000e+03	7621.000000	7.621000e+03	7.621000e+03	7.461000e+03	7.461000e+03
mean	9.188171e+06	4.792358e+05	2282.872064	7.414415e+06	1.773755e+06	3.620156e+06	3.168416e+06
std	3.746180e+07	1.911511e+06	7275.973730	2.995209e+07	7.570382e+06	1.737938e+07	1.515310e+07
min	7.000000e+00	0.000000e+00	0.000000	7.000000e+00	0.000000e+00	0.000000e+00	2.000000e+00
25%	1.356570e+05	6.004000e+03	69.000000	1.166320e+05	1.283100e+04	5.655500e+04	5.210700e+04
50%	8.182020e+05	4.547000e+04	597.000000	6.614590e+05	1.388180e+05	3.897850e+05	3.342380e+05
75%	6.625243e+06	3.428690e+05	1708.000000	5.387805e+06	1.166434e+06	2.735777e+06	2.561513e+06
max	5.132284e+08	3.501031e+07	73933.000000	4.001504e+08	1.130780e+08	2.701636e+08	2.395186e+08

8 rows × 22 columns



```
In [61]: dose1 = vaccine_data.groupby("State")[["First Dose Administered"]].max()  
dose1
```

Out[61]:

First Dose Administered	
State	
Andaman and Nicobar Islands	216046.0
Andhra Pradesh	17628583.0
Arunachal Pradesh	692475.0
Assam	10495293.0
Bihar	23350171.0
Chandigarh	700285.0
Chhattisgarh	9181482.0
Dadra and Nagar Haveli and Daman and Diu	584370.0
Delhi	7835546.0
Goa	1094392.0
Gujarat	28101222.0
Haryana	10086831.0
Himachal Pradesh	4249849.0
India	400150406.0
Jammu and Kashmir	5318516.0
Jharkhand	8382280.0
Karnataka	25847691.0
Kerala	15670747.0
Ladakh	188699.0
Lakshadweep	51156.0
Madhya Pradesh	29723036.0
Maharashtra	35040812.0
Manipur	1159424.0
Meghalaya	938572.0
Mizoram	654946.0
Nagaland	632120.0
Odisha	13954592.0
Puducherry	601591.0
Punjab	8005636.0
Rajasthan	27008606.0
Sikkim	497851.0
Tamil Nadu	20836674.0
Telangana	11649268.0
Tripura	2411195.0
Uttar Pradesh	45932488.0
Uttarakhand	5070544.0

First Dose Administered

State

West Bengal	23257417.0
-------------	------------



```
In [62]: dose2 = vaccine_data.groupby("State")[["Second Dose Administered"]].max()  
dose2
```

Out[62]:

Second Dose Administered	
State	
Andaman and Nicobar Islands	94597.0
Andhra Pradesh	6214312.0
Arunachal Pradesh	186619.0
Assam	2208577.0
Bihar	4484768.0
Chandigarh	223534.0
Chhattisgarh	2587695.0
Dadra and Nagar Haveli and Daman and Diu	80851.0
Delhi	3000536.0
Goa	302519.0
Gujarat	9051153.0
Haryana	2923550.0
Himachal Pradesh	1382592.0
India	113077994.0
Jammu and Kashmir	1489826.0
Jharkhand	1996014.0
Karnataka	7432852.0
Kerala	6426984.0
Ladakh	70337.0
Lakshadweep	17139.0
Madhya Pradesh	5733640.0
Maharashtra	12112554.0
Manipur	246694.0
Meghalaya	231982.0
Mizoram	206773.0
Nagaland	159388.0
Odisha	4200094.0
Puducherry	151771.0
Punjab	2285629.0
Rajasthan	8375056.0
Sikkim	151538.0
Tamil Nadu	4686034.0
Telangana	3965624.0
Tripura	804099.0
Uttar Pradesh	8515236.0
Uttarakhand	1596572.0

Second Dose Administered

State

West Bengal	9132961.0
-------------	-----------



```
In [63]: male = vaccine_data.groupby("State")[["Male(Individuals Vaccinated)"]].max()  
male
```

Out[63]:

Male(Individuals Vaccinated)	
State	
Andaman and Nicobar Islands	NaN
Andhra Pradesh	NaN
Arunachal Pradesh	NaN
Assam	NaN
Bihar	NaN
Chandigarh	NaN
Chhattisgarh	NaN
Dadra and Nagar Haveli and Daman and Diu	NaN
Delhi	NaN
Goa	NaN
Gujarat	NaN
Haryana	NaN
Himachal Pradesh	NaN
India	134941971.0
Jammu and Kashmir	NaN
Jharkhand	NaN
Karnataka	NaN
Kerala	NaN
Ladakh	NaN
Lakshadweep	NaN
Madhya Pradesh	NaN
Maharashtra	NaN
Manipur	NaN
Meghalaya	NaN
Mizoram	NaN
Nagaland	NaN
Odisha	NaN
Puducherry	NaN
Punjab	NaN
Rajasthan	NaN
Sikkim	NaN
Tamil Nadu	NaN
Telangana	NaN
Tripura	NaN
Uttar Pradesh	NaN
Uttarakhand	NaN

Male(Individuals Vaccinated)

State

West Bengal	NaN
-------------	-----



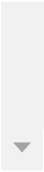
```
In [64]: female = vaccine_data.groupby("State")["Female(Individuals Vaccinated)"].max()  
female
```

Out[64]:

Female(Individuals Vaccinated)	
State	
Andaman and Nicobar Islands	NaN
Andhra Pradesh	NaN
Arunachal Pradesh	NaN
Assam	NaN
Bihar	NaN
Chandigarh	NaN
Chhattisgarh	NaN
Dadra and Nagar Haveli and Daman and Diu	NaN
Delhi	NaN
Goa	NaN
Gujarat	NaN
Haryana	NaN
Himachal Pradesh	NaN
India	115668447.0
Jammu and Kashmir	NaN
Jharkhand	NaN
Karnataka	NaN
Kerala	NaN
Ladakh	NaN
Lakshadweep	NaN
Madhya Pradesh	NaN
Maharashtra	NaN
Manipur	NaN
Meghalaya	NaN
Mizoram	NaN
Nagaland	NaN
Odisha	NaN
Puducherry	NaN
Punjab	NaN
Rajasthan	NaN
Sikkim	NaN
Tamil Nadu	NaN
Telangana	NaN
Tripura	NaN
Uttar Pradesh	NaN
Uttarakhand	NaN

Female(Individuals Vaccinated)

State
West Bengal
NaN



```
In [65]: trans = vaccine_data.groupby("State")[["Transgender(Individuals Vaccinated)"]].max()  
trans
```

Out[65]:

Transgender(Individuals Vaccinated)	
State	
Andaman and Nicobar Islands	NaN
Andhra Pradesh	NaN
Arunachal Pradesh	NaN
Assam	NaN
Bihar	NaN
Chandigarh	NaN
Chhattisgarh	NaN
Dadra and Nagar Haveli and Daman and Diu	NaN
Delhi	NaN
Goa	NaN
Gujarat	NaN
Haryana	NaN
Himachal Pradesh	NaN
India	46462.0
Jammu and Kashmir	NaN
Jharkhand	NaN
Karnataka	NaN
Kerala	NaN
Ladakh	NaN
Lakshadweep	NaN
Madhya Pradesh	NaN
Maharashtra	NaN
Manipur	NaN
Meghalaya	NaN
Mizoram	NaN
Nagaland	NaN
Odisha	NaN
Puducherry	NaN
Punjab	NaN
Rajasthan	NaN
Sikkim	NaN
Tamil Nadu	NaN
Telangana	NaN
Tripura	NaN
Uttar Pradesh	NaN
Uttarakhand	NaN

State

West Bengal

NaN

In []: